## ABSTRACT

This is the report of the METU EE314 Course 2014-2015 Spring term project. For the project, five games described in [2] are offered for implementation with FPGA, Verilog HDL and VGA interface. The game chosen is 'Dipole'. The report explains logical flow of the design and different aspects of the work done.


## INTRODUCTION

The game Dipole is picked for the project. Dipole is a game in which a user controls 2 dots on a circle turning clockwise or counterclockwise, trying not to collide obstacles flowing downwards. Dots have exactly $180^o$ in between throughout the game. When a dot collides an obstacle, the game ends. The number of the blocks been passed successfully is the score.

In addition to basic requirements, our design includes additional features which will be explained later. Among them, we think the most important is that obstacles have four behavior types. These types are simple, rotating, horizontally moving and disappearing.

The project expects an implementation on Altera De1 FPGA offered by the METU EEE Department, to control VGA interface using Verilog HDL. We have been got used to using the FPGA and its software elements throughout the term in EE314 Course. A presentation explaining basic aspects of Verilog HDL was performed during the term by EE348 assistants. For detailed information and work, we got help from [4] . The information about VGA interface is obtained from the document [3] supplied with by course staff. Simulation was very important for this project since we had not much opportunity to work on the FPGA offered to common usage by the department. A VGA interface simulation program named VGASim was found, learnt how to use and used.

More detailed explanations will be done step by step including flow charts, code analyze, photos.

## DESIGN

The design of the project includes the features below besides fundamental requirements such as VGA synchronization, dots, obstacles and so on. These features are basically listed here, and will explained in detail later.

- The game has four states, namely; start screen, gameplay, pause, end screen,
- There is 'game level' counting up till five with time. The game operates accordingly in the aspect of speed and obstacle type,
- Time passed, score and level are hold and shown,
- High score is shown at start screen every time; and if a game ends with a high score, it is informed at the end screen,
- Specifications of a block are decided randomly with the help of a random algorithm,
- Blocks have three specifications; behavior, size and horizontal position.
    - 4 behavior type; simply falling, rotating, horizontally moving, disappearing,
    - 3 size type; square, long, short,
    - 3 horizontal position; left, right, middle,

- Random instruction created is processed before block generation in order to:
    - Prevent undesired combinations (for example; a square rotating obstacle)
    - Prevent a block with difficult behavior to come at low levels; to make the game more difficult at higher levels, easy at lower levels,
    - Prevent the fail of randomness (when three successive blocks with the same horizontal position are sensed, seed of the random algorithm is changed)
- Debouncing for start and pause buttons,
- When dot controlling buttons pressed together, the latest pressed button acts

The Verilog design consists of 12 modules.

1. Game (main module)
2. Character
3. Letter
4. Letters
5. Random
6. Rotatedot
7. Statedecider
8. Timecounter
9. Behavioralblock
10. Blockmaster
11. Scorecounter
12. Highscore

They are in the best order for understanding in aspect of interactions between them. They will be explained below in this order. For each of them, their aim and behavioral operation will be explained adequately firstly. Next, the Verilog code will be supplied and more detailed explanation will be given as comment wherever needed.

1. Game (Main Module)

It is the main module of the project. All other modules are called here. It receives internal 50Mhz clock of the FPGA and 4 buttons as input and gives output to HS, VS, R, G, B and clock pins of the VGA interface.

VGA Synchronization: Firstly, 25 MHz clock signal is generated out of 50 MHz one. With 25MHz clock *clock_screen*, *xcount* counts up to 800. *ycount* increment whenever *xcount* ends the cycle up to 525. *clk_frame* makes an impulse when *ycount* turns to 0. That is, the processes clocked with positive edge of *clk_frame* are done at every frame completion instant. The Figure 1 explains how to give values to HS and VS of the VGA interface. On the figure, one should think that the monitoring process starts from top-left corner, proceed to the right with every positive edge of 25Mhz clock, and when it is at the right-most, turns to the left-most and starts from the row just below. When it is considered as a loop, it perfectly fits the VGA synchronization explanations. The visible screen corresponds to the area shown in the figure. From the *xcount* and *ycount* values, the horizontal and vertical parameters of the visible screen is generated an held in *x* and *y* variables. All the displaying operations will use these parameters.

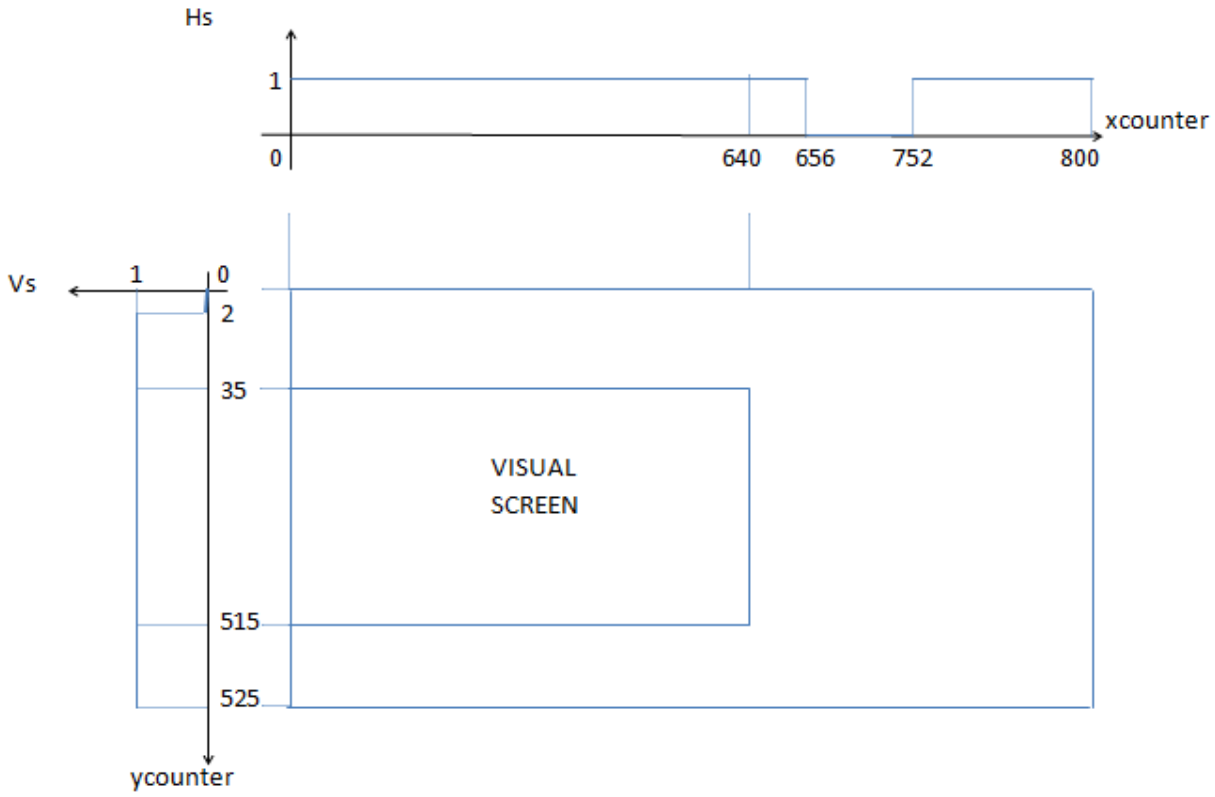


Figure 1: VGA Synchronization

Object Displaying Method: Our method is using a single bit wire for every object which is a result of a logical function of variables *x*, *y* (the position of the monitoring operation) and desired positions. To explain this, I will use the *fon* wire in the module 'Game' and the figure 2. *fon* variable represents the yellow rectangle at the back of the "Dipole" text at the start screen.

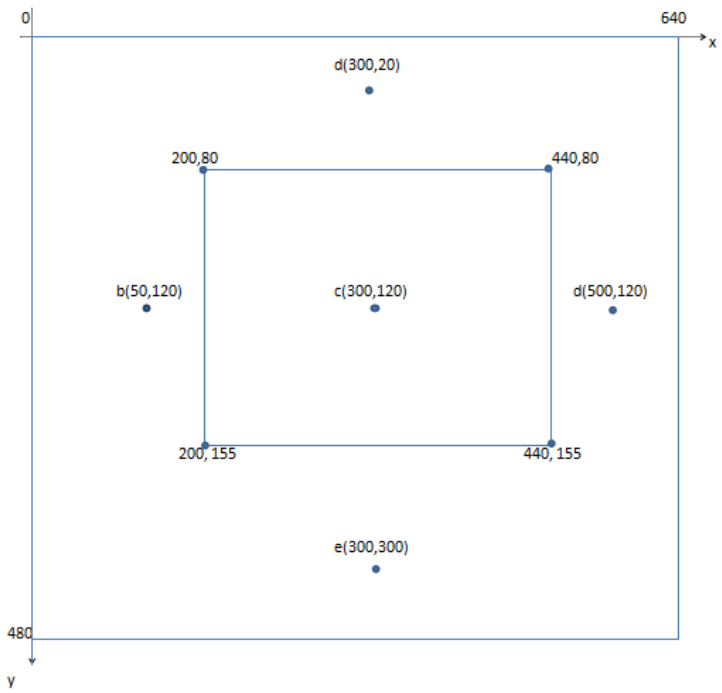


Figure 2: Displaying Method Example

```
assign fon = (x>200)&&(x<440)&&(y>80)&&(y<155);
```

*x* and *y* are the position of the monitoring process. That is, *x* takes all the values 0 and 640, and *y* takes all the values between 0 and 480. When the monitor is coloring the pixels *fon*'s borders, *fon* gets high. For example, monitor will color a, b, c, d, e points in the Figure 2 respectively. Only when it is on the point c, *fon* will be high. Then, at another place in the module 'Game' where we call "Color Giving Block", if the *fon* is high and other conditions are as desired, *color* (outputting R, G, B pins of the VGA interface) is attained to desired values representing desired colors. All objects are displayed in this way.

In "Color Giving Block", objects are assessed according to states and some conditions for game play. For example, modules generating object variables for the letters of the text "Game Over" work all the time but the text only colored at the start screen.

Sine-Cosine Generation: The function $f(x) = 1024 * (1 + \cos(2x))$ is written in a look up table between 0 and 90 degrees and in 2 degrees steps. Then the $f(x) = 1024 * (1 + \cos(2x))$ and $g(x) = 1024 * (1 + \sin(2x))$ functions are generated out of the look up table in 2 degrees steps and held in *cosdot*, *sindot* wires. As the parameter *dotangle* is used. For example; *cosdot*=1204*(1+sin(2**dotangle*)).The reason to use these type of functions is preventing the negative and floating numbers. One should examine the expressions in which *sindot* and *cosdot* used remembering they are not directly sine and cosine functions.

Since dot positions are defined accordingly, to turn dots, *dotangle* is incremented or decremented. Rotating operation is done in this module.

Debouncing: Pause and start buttons are debounced, and in the rest of the program, debounced variables are used. When a button pushed, debounced variable gets high for one frame duration. Then it gets low and unless the button stays unpressed for 7 frame passes, the debounced variable is not permitted to get high again. The debouncing corresponds to nearly 120 milliseconds.

Also, in the module, the successive block positions are controlled for the sake of randomness with *change1* and *change2* variables as it is explained later. Any collision is inspected. Also, time and score panels change position according to the state. While they are at top-left in game play, they come to the middle at the end game.

```
module game (clk, pausebutton, startbutton, ccwbutton, cwbutton, hs, vs, r, g,
b, clk_screen);
input clk; // 50 MHz internal clock of the FPGA
input pausebutton, startbutton, ccwbutton, cwbutton; // buttons (active low)
output hs, vs, r, g, b, clk_screen; // VGA outputs
reg clk_screen=1'b0, clk_frame=1'b0; // 25MHz and (nearly) 60Hz generated
clocks
reg pausebuttondb =1'b0, startbuttondb =1'b0; // debounced and active high
buttons
reg collision=1'b0, hs=1'b0, vs=1'b0;
reg[2:0] pausecounter =3'b000, startcounter =3'b000; // used for debouncing
reg[2:0] color=3'b000; // its bits will be connected to r, g, b outputs
respectively
reg [8:0] dotangle=8'd0; //
reg[10:0] xcount=10'd0; //holds horizontal VGA process(screen(640)
//+forthporch(16)+hs(96)+backporch(48))
reg[10:0] ycount=10'd1; //holds vertical VGA
process(screen(480)+forthporch(10)+vs(2)+backporch(33))
integer sinusoidal[0:45];// nth entry corresponds to 1024*(1+cos(2n)) for
n=0..45
wire r, g, b; // VGA outputs
wire backcircle, dot1, dot2, block1, block2,
wire block3, block4, pauseindicator ; // This block of wires all
wire char1, char2, charsc, char3, char4,
wire char5, char6, char7, char8, char9; // represent different visual
wire char10, char11, char12, go_g, go_a,
wire go_m, go_e, go_o, go_v, go_e2, go_r; // objects. When the monitor
wire dipole_d, dipole_i, dipole_p,
wire dipole_o, dipole_l, dipole_e; // comes to the pixels that the
wire high_h, high_i, high_g, high_h2,
wire high_sc, lev_l, lev_s; // object is desired to take place;
wire high2_h, high2_i, high2_g, high2_h2,
wire high2_s, high2_c, high2_o, high2_r; // the wire becomes '1', otherwise
'0'.
wire high2_e, high2_donk; // Later, we give rgb a value
wire score_s, score_c, score_o, score_r, score_e, score_sc;// accordingly in
order to give a
wire time_t ,time_i ,time_m ,time_e ,time_sc; // color the object.
wire newrequest1, newrequest2, newrequest3,
wire newrequest4, yend1, yend2; // connections between modules
wire yend3, yend4, enable1, enable2, enable3,
wire enable4, score1, score2; // "behavioralblock" and "blockmaster"
wire score3, score4
wire fade1, fade2, fade3, fade4; // output of "behavioralblock". Block
dissappears accordingly
```

```
wire highhappened ; //in this module output of "highscore". Whether a game
ended with a high score.
wire cw, ccw; //At the end game screen, high score is informed if it's '1'.
wire change1, change2; //output of "rotatedot". Dots rotate accordingly.
//Whether 3 successive blocks have same horizontal position.
//Random seed is changed accordingly.
wire[3:0] min1, min0, sec1, sec0, s3, s2, s1, s0, shigh3; // Some outputs
wire[3:0] shigh2, shigh1, shigh0, difficulty, speed; // and inputs of modules
wire[6:0] randomnumber ; // and connections
wire[31:0] scorex3,scorex2,scorex1,scorex0,scorey; // in between. All
wire[31:0] timex3, timex2, timesc, timex1, timex0, timey; // will be
wire[31:0] posxblock1, posxblock2, posxblock3, posxblock4; // explained
wire[1:0] state; // describes 4 states of the game (detail : state module)
wire[31:0] x, y; // holds the position of the visible monitoring process
// (screen portion of the xcount and ycount)
wire[31:0] sindot, cosdot; // sindot gives 1024*(1+sin(2*dotangle)).
// It is defined on 360 degree in 2 degree steps
assign change1= (posxblock1 == posxblock2) && (posxblock1 == posxblock3)
&& (posxblock1 == 320) && (posxblock1 == 265);
assign change2= (posxblock1 == posxblock2) && (posxblock1 == posxblock3)
&& (posxblock1 == 375);
// If 3 blocks have same horizontal position, one of them give '1'.
//Seed of the random algorithm changes accordingly.
assign sindot = (dotangle<46)?sinusoidal[45-dotangle]
:(dotangle>45 && dotangle<91)?sinusoidal[dotangle-45]
:(dotangle>90 && dotangle<136)?(2048-sinusoidal[135-dotangle])
:(dotangle>135 && dotangle<180)?(2048-sinusoidal[dotangle-135]):31'bx;
assign cosdot = (dotangle<46)?sinusoidal[dotangle]:(dotangle>45 &&
dotangle<91)
?(2048-sinusoidal[90-dotangle])
:(dotangle>90 && dotangle<136)?(2048-sinusoidal[dotangle-90])
:(dotangle>135 && dotangle<180)?sinusoidal[180-dotangle]:31'bx;
// for example;sindot gives 1024*(1+sin(2*dotangle)).It is defined on 360
degree in 2 degree steps
assign x=(xcount<640)?(xcount+1):((xcount==(800))?1:0);
assign y=(ycount>35 && ycount<516 && xcount<(800))?(ycount-35)
:(ycount>34 && ycount<515 && xcount==(800))?(ycount-34):0;
// they hold the position of the visible monitoring process
//(screen portion of the xcount and ycount).All visual object placements will
use these parameters
assign fon = (x>200)&&(x<440)&&(y>80)&&(y<155); // the yellow portion at the
back of
//the DIPOLE text at start screen
assign pauseindicator = ((x<310 && x>280)
|| (x<360 && x>330)) && (y<250 && y>150); // the universal pause indicator
seen
assign backcircle = ((x-320)**2 + (y-384)**2)<(76**2+1)//at pause screen
&& ((x-320)**2 + (y-384)**2)>(74**2-1); // Predefined ring under dots.The
geometrical ring
// equation for a circle with center
//(320,384) and 74<=r<=76
assign dot1 = ((x-(75*cosdot >> 10)-245)**2 + (y+(75*sindot >> 10)-
459)**2)<(9**2+1);
// equation for a circle with center (320+75cosx,384-75sinx) and r<=9
assign dot2 = ((x+(75*cosdot >> 10)-395)**2 + (y-(75*sindot >> 10)-
309)**2)<(9**2+1);
// equation for a circle with center (320-75cosx,384+75sinx) and r<=9
```

```
assign r=color[2]; // In order to give
assign g=color[1]; // colors on a single
assign b=color[0]; // variable 'color'
assign scorex3 = (state==2'b11)?340:20; // state 11 is end game.
assign scorex2 = (state==2'b11)?360:40; // This block aims to
assign scorex1 = (state==2'b11)?380:60; // bring "Score Panel" and
assign scorex0 = (state==2'b11)?400:80; // "Time Panel" to somewhere in middle
assign scorey = (state==2'b11)?250:60; // at the end of the game.
assign timex3 = (state==2'b11)?340:20; // At gameplay and pause states
assign timex2 = (state==2'b11)?360:40; // which are denoted with 01 and 10;
assign timesc = (state==2'b11)?380:60; // they are at top-left of
assign timex1 = (state==2'b11)?400:80; // the screen. At starting screen,
assign timex0 = (state==2'b11)?420:100; // they are not given color at "Color
Giving Block" below;
assign timey = (state==2'b11)?280:20; // hence they are not seen.
assign speed= (difficulty<3)?4'd2:(difficulty<5)?4'd3:4'd4;
// speed will be increment in y position and change in angles of dots and
blocks while rotating.
character u0 (min1,x,y,timex3,timey,char1); // Time Panel
character u1 (min0,x,y,timex2,timey,char2); //
letters usc (5'd26,x,y,timesc,timey,charsc); //
character u2 (sec1,x,y,timex1,timey,char3); //
character u3 (sec0,x,y,timex0,timey,char4); //
character sc3 (s3,x,y,scorex3,scorey,char8); // Score Panel
character sc2 (s2,x,y,scorex2,scorey,char7); //
character sc1 (s1,x,y,scorex1,scorey,char6); //
character sc0 (s0,x,y,scorex0,scorey,char5); //
letters lev1 (5'd11,x,y,20,100,lev_l); // Level Panel
character lev2 (difficulty,x,y,40,100,lev_s); //
letter lend1 (5'd6,x,y,255,70,go_g); // "Game Over" text
letter lend2 (5'd0,x,y,290,70,go_a); //
letter lend3 (5'd12,x,y,325,70,go_m); //
letter lend4 (5'd4,x,y,360,70,go_e); //
letter lend5 (5'd14,x,y,255,120,go_o); //
letter lend6 (5'd21,x,y,290,120,go_v); //
letter lend7 (5'd4,x,y,325,120,go_e2); //
letter lend8 (5'd17,x,y,360,120,go_r); //
letters lend9 (5'd7,x,y,210,200,high2_h); // "High Score!" Text
letters lend10 (5'd8,x,y,230,200,high2_i); // which appears only if
letters lend11 (5'd6,x,y,250,200,high2_g); // the game ended with
letters lend12 (5'd7,x,y,270,200,high2_h2); // a high score. Whether it
letters lend13 (5'd18,x,y,310,200,high2_s); // appears or not is
letters lend14 (5'd2,x,y,330,200,high2_c); // decided at "Color Giving
letters lend15 (5'd14,x,y,350,200,high2_o); // Block" below
letters lend16 (5'd17,x,y,370,200,high2_r); //
letters lend17 (5'd4,x,y,390,200,high2_e); //
letters lend18 (5'd27,x,y,410,200,high2_donk);//
letters lend19 (5'd18,x,y,200,250,score_s); // "Score:" Text
letters lend20 (5'd2,x,y,220,250,score_c); // at the end screen.
letters lend21 (5'd14,x,y,240,250,score_o); // "Score Panel" will come near it
letters lend22 (5'd17,x,y,260,250,score_r); // as adjusted above
letters lend23 (5'd4,x,y,280,250,score_e); //
letters lend24 (5'd26,x,y,310,250,score_sc); //
letters lend25 (5'd19,x,y,220,280,time_t); // "Time:" Text
letters lend26 (5'd8,x,y,240,280,time_i); // at the end screen.
letters lend27 (5'd12,x,y,260,280,time_m); // "Time Panel" will come near it
letters lend28 (5'd4,x,y,280,280,time_e); // as adjusted above
```

```
letters lend29 (5'd26,x,y,310,280,time_sc); //
letter lstart1 (5'd3,x,y,235,100,dipole_d); // "Dipole" Text
letter lstart2 (5'd8,x,y,265,100,dipole_i); //
letter lstart3 (5'd15,x,y,295,100,dipole_p); //
letter lstart4 (5'd14,x,y,325,100,dipole_o); //
letter lstart5 (5'd11,x,y,355,100,dipole_l); //
letter lstart6 (5'd4,x,y,385,100,dipole_e); //
letters lstart7 (5'd7,x,y,235,180,high_h); // "High:" Text
letters lstart8 (5'd8,x,y,255,180,high_i); //
letters lstart9 (5'd6,x,y,275,180,high_g); //
letters lstart10 (5'd7,x,y,295,180,high_h2); //
letters lstart11 (5'd26,x,y,312,180,high_sc); //
character hsc0 (shigh0,x,y,390,180,char9); // High Score Panel
character hsc1 (shigh1,x,y,370,180,char10); // near "High:" Text
character hsc2 (shigh2,x,y,350,180,char11); //
character hsc3 (shigh3,x,y,330,180,char12); //
// Other modules. Detailed information will be given inside every module.
timecounter t0 (clk_frame, state, min1, min0, sec1, sec0, difficulty);
scorecounter sc (clk_frame, state, score1, score2, score3, score4, s3, s2, s1,
s0);
highscore hsc (clk_frame,s3,s2,s1,s0,shigh3,shigh2,shigh1,shigh0, highhappened
);
random r0 (clk_frame, change1, change2, randomnumber );
blockmaster bm0 (clk_frame, state, newrequest1, newrequest2, newrequest3,
newrequest4,
yend1, yend2, yend3, yend4, enable1, enable2, enable3, enable4);
behavioralblock b1 (x, y, clk_frame, enable1, state, randomnumber ,
difficulty,
posxblock1, newrequest1, yend1, score1, fade1, block1);
behavioralblock b2 (x, y, clk_frame, enable2, state, randomnumber ,
difficulty,
posxblock2, newrequest2, yend2, score2, fade2, block2);
behavioralblock b3 (x, y, clk_frame, enable3, state, randomnumber ,
difficulty,
posxblock3, newrequest3, yend3, score3, fade3, block3);
behavioralblock b4 (x, y, clk_frame, enable4, state, randomnumber ,
difficulty,
posxblock4, newrequest4, yend4, score4, fade4, block4);
rotatedot ro (clk_frame,ccwbutton, cwbutton, ccw, cw);
statedecider sd (clk_frame, startbuttondb , pausebuttondb , collision, state);
initial begin
sinusoidal[0]=2048;
sinusoidal[1]=2047;
sinusoidal[2]=2046;
sinusoidal[3]=2042;
sinusoidal[4]=2038;
sinusoidal[5]=2032;
sinusoidal[6]=2026;
sinusoidal[7]=2018;
sinusoidal[8]=2008;
sinusoidal[9]=1998;
sinusoidal[10]=1986;
sinusoidal[11]=1973;
sinusoidal[12]=1959;
sinusoidal[13]=1944;
sinusoidal[14]=1928;
sinusoidal[15]=1911;
```

```
sinusoidal[16]=1892;
sinusoidal[17]=1873;
sinusoidal[18]=1852;
sinusoidal[19]=1831;
sinusoidal[20]=1808;
sinusoidal[21]=1785;
sinusoidal[22]=1761;
sinusoidal[23]=1735;
sinusoidal[24]=1709;
sinusoidal[25]=1682;
sinusoidal[26]=1654;
sinusoidal[27]=1626;
sinusoidal[28]=1597;
sinusoidal[29]=1567;
sinusoidal[30]=1536;
sinusoidal[31]=1505;
sinusoidal[32]=1473;
sinusoidal[33]=1440;
sinusoidal[34]=1408;
sinusoidal[35]=1374;
sinusoidal[36]=1340;
sinusoidal[37]=1306;
sinusoidal[38]=1272;
sinusoidal[39]=1237;
sinusoidal[40]=1202;
sinusoidal[41]=1167;
sinusoidal[42]=1131;
sinusoidal[43]=1095;
sinusoidal[44]=1060;
sinusoidal[45]=1024;
end
always @ (posedge clk) // 25 MHz clock generation. clk_screen is 25MHz, clk is
50 Mhz
begin
clk_screen=!clk_screen;
end
always @ (posedge clk_screen) // Consists of "VGA Synch Block", "Color Giving
Block"
begin // and "Collision Informer Block".Note that this portion
//operates with 25MHz clock. That is, the "VGA Synch Block"
xcount=xcount+1;
if (xcount>(800))
begin
xcount=1;
ycount=ycount+1;
end
if (ycount>525)
begin
ycount=1;
clk_frame=1;
end
else clk_frame=0;
if (xcount<657 || xcount>752) hs=1;
else hs=0;
if (ycount<3) vs=0;
else vs=1;
// "Color Giving Block"
```

```
if (x && y)
begin
if (state==2'b00) // At Start Screen
color=(high_h | high_i | high_g | high_h2 | high_sc | char9 | char10 | char11
| char12)?3'b001
:(dipole_d | dipole_i | dipole_p | dipole_o | dipole_l | dipole_e)?3'b001
:(fon)?3'b110
:(dot1)?3'b100:(dot2)?3'b110
:(backcircle)?3'b001:3'b000;
if (state==2'b01 || state==2'b10) // At Pause and Gameplay Screen
color=(pauseindicator && state==2'b10)?3'b010
:(char1 || char2 || charsc || char3 || char4 || char5 || char6 || char7 ||
char8
|| lev_l || lev_s)?3'b111
:(dot1)?3'b100:(dot2)?3'b110
:((block1 && (state==2'b11 || !fade1))| (block2 && (state==2'b11 || !fade2))
| (block3 && (state==2'b11 || !fade3)) | (block4 && (state==2'b11 ||
!fade4)))?3'b111
:(backcircle)?3'b001:3'b000;
if (state==2'b11) // At End Screen
color=(go_g | go_a | go_m | go_e | go_o | go_v | go_e2 | go_r)?3'b100
:(highhappened && (high2_h | high2_i | high2_g | high2_h2 | high2_s | high2_c
| high2_o | high2_r | high2_e | high2_donk))?3'b010
:(score_s | score_c | score_o | score_r | score_e | score_sc | time_t | time_i
| time_m | time_e | time_sc
| char1 | char2 | charsc | char3 | char4 | char5 | char6 | char7 |
char8)?3'b100
:(dot1)?3'b100:(dot2)?3'b110
:((block1 && (state==2'b11 || !fade1))| (block2 && (state==2'b11 || !fade2))
| (block3 && (state==2'b11 || !fade3)) | (block4 && (state==2'b11 ||
!fade4)))?3'b111
:(backcircle)?3'b001:3'b000;
end
else color=3'b000;
// "Collision Detector Block"
if ((dot1 || dot2) && (block1 || block2 || block3 || block4)) collision=1;
if (state==2'b00 && collision==1) collision=0;
end
always @ (posedge clk_frame)
begin
// "Dot Rotation"
if (ccw && state==2'b01)
begin
if (dotangle>(179-speed)) dotangle=dotangle+speed-180;
else dotangle=dotangle+speed;
end
if (cw && state==2'b01)
begin
if (dotangle<speed) dotangle=dotangle+180-speed;
else dotangle=dotangle-speed;
end
if (state==2'b00) dotangle=0;
// "Pause Button Debouncer"
if (!pausebutton && pausecounter ==3'b000)
begin
pausebuttondb <=1'b1;
pausecounter <=3'b001;
```

```
end
if (pausecounter !=3'b000)
begin
pausebuttondb <=1'b0;
if(pausebutton)
begin
if (pausecounter ==3'b111) pausecounter <=3'b000;
else pausecounter <=pausecounter +3'b001;
end
else pausecounter <=3'b001;
end
// "Start Button Debouncer"
if (!startbutton && startcounter ==3'b000)
begin
startbuttondb <=1'b1;
startcounter <=3'b001;
end
if (startcounter !=3'b000)
begin
startbuttondb <=1'b0;
if(startbutton)
begin
if (startcounter ==3'b111) startcounter <=3'b000;
else startcounter <=startcounter +3'b001;
end
else startcounter <=3'b001;
end
end
endmodule
```

2. Character

Well-known 7 segment display is realized in this module. The module covers 15 horizontal and 21 vertical pixels. In this range, 7 segments are defined. 10 digits are also defined in a look-up table in terms of these segments. The module can be placed anywhere in the screen thanks that it generates its own positional parameters out of general positional parameters and desired positional parameters given as input. While monitor is processing a pixel within a segment which should be active according to desired digit, the module outputs 1, otherwise 0. Later, the pixels which the output is 1 during their process are colored in the module 'Game' in "Color Giving Block". All the digits at screen are written by this module.

```
module character (in, x, y, posx, posy, out);
input[3:0] in;
input[31:0] x, y, posx, posy;
output out;
wire[31:0] xcell, ycell;
wire[1:7] line;
reg[1:7] tablo[0:9];
wire a,b,c,d,e,f,g;
initial begin
tablo[0]=7'b1111110; // defining digits in terms of segments
tablo[1]=7'b0000110;
tablo[2]=7'b1101101;
tablo[3]=7'b1111001;
tablo[4]=7'b0110011;
tablo[5]=7'b1011011;
```

```
tablo[6]=7'b1011111;
tablo[7]=7'b1110000;
tablo[8]=7'b1111111;
tablo[9]=7'b1111011;
end
assign line=tablo[in];
// inner positions for only the area the module covers
assign xcell = (x>posx && x<(posx+16) && y>posy && y<(posy+22))?(x-posx):0;
assign ycell = (x>posx && x<(posx+16) && y>posy && y<(posy+22))?(y-posy):0;
assign a = (ycell>0 && ycell<3)?1:0; // segment definitions
assign b = (xcell>13 && xcell<16 && ycell<11 && ycell>0)?1:0;
assign c = (xcell>13 && xcell<16 && ycell>10 && ycell<22)?1:0;
assign d = (ycell>19 && ycell<22)?1:0;
assign e = (xcell>0 && xcell<3 && ycell>10 && ycell<22)?1:0;
assign f = (xcell>0 && xcell<3 && ycell<11 && ycell>0)?1:0;
assign g = (ycell>9 && ycell<12)?1:0;
assign out = a&(line[1]) | b&(line[2]) | c&(line[3]) | d&(line[4])
| e&(line[5]) | f&(line[6]) | g&(line[7]); // representative one bit variable
endmodule
```

3. Letter

This module is similar to the module 'Character', but instead of 7 segments; 5 horizontal, 7 vertical, total 35 square segments are defined. Each square segment covers 5x5 pixels. The module covers 25x35 pixels. The working principle is same with the module 'Character'. The text "DIPOLE" at the start screen, and the text "GAME OVER" at the end screen are written by this module.

```
module letter (in, x, y, posx, posy, out);
input[4:0] in;
input[31:0] x, y, posx, posy;
output out;
wire[31:0] xcell, ycell;
wire[1:35] line;
reg[1:35] tablo[0:27];
wire r1,r2,r3,r4,r5,r6,r7,c1,c2,c3,c4,c5;
initial begin
tablo[0]=35'b01110_10001_10001_11111_10001_10001_10001 ; // A
tablo[1]=35'b11110_10001_10001_11110_10001_10001_11110 ; // B
tablo[2]=35'b01111_10000_10000_10000_10000_10000_01111 ; // C
tablo[3]=35'b11110_10001_10001_10001_10001_10001_11110 ; // D
tablo[4]=35'b11111_10000_10000_11110_10000_10000_11111 ; // E
tablo[5]=35'b11111_10000_10000_11110_10000_10000_10000 ; // F
tablo[6]=35'b01111_10000_10000_10111_10001_10001_01110 ; // G
tablo[7]=35'b10001_10001_10001_11111_10001_10001_10001 ; // H
tablo[8]=35'b11111_00100_00100_00100_00100_00100_11111 ; // I
tablo[9]=35'b00010_00010_00010_00010_10010_10010_01100 ; // J
tablo[10]=35'b10001_10010_10100_11000_10100_10010_10001 ; // K
tablo[11]=35'b10000_10000_10000_10000_10000_10000_11111 ; // L
tablo[12]=35'b10001_11011_10101_10101_10001_10001_10001 ; // M
tablo[13]=35'b10001_10001_11001_10101_10011_10001_10001 ; // N
tablo[14]=35'b01110_10001_10001_10001_10001_10001_01110 ; // O
tablo[15]=35'b11110_10001_10001_11110_10000_10000_10000 ; // P
tablo[16]=35'b01110_10001_10001_10001_10101_10011_01111 ; // Q
tablo[17]=35'b11110_10001_10001_11110_10100_10010_10001 ; // R
tablo[18]=35'b01111_10000_10000_11110_00001_00001_11110 ; // S
tablo[19]=35'b11111_00100_00100_00100_00100_00100_00100 ; // T
```

```
tablo[20]=35'b10001_10001_10001_10001_10001_10001_01110 ; // U
tablo[21]=35'b10001_10001_10001_10001_10001_01010_00100 ; // V
tablo[22]=35'b10001_10001_10001_10101_10101_11011_10001 ; // W
tablo[23]=35'b10001_10001_01010_00100_01010_10001_10001 ; // X
tablo[24]=35'b10001_10001_01010_00100_00100_00100_00100 ; // Y
tablo[25]=35'b11111_00001_00010_00100_01000_10000_11111 ; // Z
tablo[26]=35'b00000_00000_00000_00100_00000_00100_00000 ; // :
tablo[27]=35'b10000_10000_10000_10000_10000_00000_10000 ; // !
end
assign line=tablo[in];
// inner positions
assign xcell = (x>posx && x<(posx+26) && y>posy && y<(posy+36))?(x-posx):0; //
25x35
assign ycell = (x>posx && x<(posx+26) && y>posy && y<(posy+36))?(y-posy):0;
// rows
assign r1 = (ycell>0 && ycell<6)?1:0; // 5x5 squares
assign r2 = (ycell>5 && ycell<11)?1:0;
assign r3 = (ycell>10 && ycell<16)?1:0;
assign r4 = (ycell>15 && ycell<21)?1:0;
assign r5 = (ycell>20 && ycell<26)?1:0;
assign r6 = (ycell>25 && ycell<31)?1:0;
assign r7 = (ycell>30 && ycell<36)?1:0;
//columns
assign c1 = (xcell>0 && xcell<6)?1:0;
assign c2 = (xcell>5 && xcell<11)?1:0;
assign c3 = (xcell>10 && xcell<16)?1:0;
assign c4 = (xcell>15 && xcell<21)?1:0;
assign c5 = (xcell>20 && xcell<26)?1:0;
// Note that the first operation is bitwise and. For Example
// the semi column above (26. entry). It lights only when
// the monitoring is at c3&r4 or c3&r6
assign out = (((line[1:5] & {c1,c2,c3,c4,c5}) != 5'd0) && r1)
|| (((line[6:10] & {c1,c2,c3,c4,c5}) != 5'd0) && r2)
|| (((line[11:15] & {c1,c2,c3,c4,c5}) != 5'd0) && r3)
|| (((line[16:20] & {c1,c2,c3,c4,c5}) != 5'd0) && r4)
|| (((line[21:25] & {c1,c2,c3,c4,c5}) != 5'd0) && r5)
|| (((line[26:30] & {c1,c2,c3,c4,c5}) != 5'd0) && r6)
|| (((line[31:35] & {c1,c2,c3,c4,c5}) != 5'd0) && r7);
      endmodule
```

4. Letters

This module is the same with the module 'Letter', but square segments cover 3x3 pixels. The module covers 15x21 pixels in total. All the texts except the ones written with the module 'Letter' are written by this module.

```
module letters (in, x, y, posx, posy, out);
input[4:0] in;
input[31:0] x, y, posx, posy;
output out;
wire[31:0] xcell, ycell;
wire[1:35] line;
reg[1:35] tablo[0:27];
wire r1,r2,r3,r4,r5,r6,r7,c1,c2,c3,c4,c5;
initial begin
tablo[0]=35'b01110_10001_10001_11111_10001_10001_10001 ; // A
```

```
tablo[1]=35'b11110_10001_10001_11110_10001_10001_11110 ; // B
tablo[2]=35'b01111_10000_10000_10000_10000_10000_01111 ; // C
tablo[3]=35'b11110_10001_10001_10001_10001_10001_11110 ; // D
tablo[4]=35'b11111_10000_10000_11110_10000_10000_11111 ; // E
tablo[5]=35'b11111_10000_10000_11110_10000_10000_10000 ; // F
tablo[6]=35'b01111_10000_10000_10111_10001_10001_01110 ; // G
tablo[7]=35'b10001_10001_10001_11111_10001_10001_10001 ; // H
tablo[8]=35'b11111_00100_00100_00100_00100_00100_11111 ; // I
tablo[9]=35'b00010_00010_00010_00010_10010_10010_01100 ; // J
tablo[10]=35'b10001_10010_10100_11000_10100_10010_10001 ; // K
tablo[11]=35'b10000_10000_10000_10000_10000_10000_11111 ; // L
tablo[12]=35'b10001_11011_10101_10101_10001_10001_10001 ; // M
tablo[13]=35'b10001_10001_11001_10101_10011_10001_10001 ; // N
tablo[14]=35'b01110_10001_10001_10001_10001_10001_01110 ; // O
tablo[15]=35'b11110_10001_10001_11110_10000_10000_10000 ; // P
tablo[16]=35'b01110_10001_10001_10001_10101_10011_01111 ; // Q
tablo[17]=35'b11110_10001_10001_11110_10100_10010_10001 ; // R
tablo[18]=35'b01111_10000_10000_11110_00001_00001_11110 ; // S
tablo[19]=35'b11111_00100_00100_00100_00100_00100_00100 ; // T
tablo[20]=35'b10001_10001_10001_10001_10001_10001_01110 ; // U
tablo[21]=35'b10001_10001_10001_10001_10001_01010_00100 ; // V
tablo[22]=35'b10001_10001_10001_10101_10101_11011_10001 ; // W
tablo[23]=35'b10001_10001_01010_00100_01010_10001_10001 ; // X
tablo[24]=35'b10001_10001_01010_00100_00100_00100_00100 ; // Y
tablo[25]=35'b11111_00001_00010_00100_01000_10000_11111 ; // Z
tablo[26]=35'b00000_00000_00000_00100_00000_00100_00000 ; // :
tablo[27]=35'b10000_10000_10000_10000_10000_00000_10000 ; // !
end
assign line=tablo[in];
assign xcell = (x>posx && x<(posx+16) && y>posy && y<(posy+22))?(x-posx):0;
//15x21
assign ycell = (x>posx && x<(posx+16) && y>posy && y<(posy+22))?(y-posy):0;
assign r1 = (ycell>0 && ycell<4)?1:0; // the squares are 3x3
assign r2 = (ycell>3 && ycell<7)?1:0;
assign r3 = (ycell>6 && ycell<10)?1:0;
assign r4 = (ycell>9 && ycell<13)?1:0;
assign r5 = (ycell>12 && ycell<16)?1:0;
assign r6 = (ycell>15 && ycell<19)?1:0;
assign r7 = (ycell>18 && ycell<22)?1:0;
assign c1 = (xcell>0 && xcell<4)?1:0;
assign c2 = (xcell>3 && xcell<7)?1:0;
assign c3 = (xcell>6 && xcell<10)?1:0;
assign c4 = (xcell>9 && xcell<13)?1:0;
assign c5 = (xcell>12 && xcell<16)?1:0;
assign out = (((line[1:5] & {c1,c2,c3,c4,c5}) != 5'd0) && r1)
|| (((line[6:10] & {c1,c2,c3,c4,c5}) != 5'd0) && r2)
|| (((line[11:15] & {c1,c2,c3,c4,c5}) != 5'd0) && r3)
|| (((line[16:20] & {c1,c2,c3,c4,c5}) != 5'd0) && r4)
|| (((line[21:25] & {c1,c2,c3,c4,c5}) != 5'd0) && r5)
|| (((line[26:30] & {c1,c2,c3,c4,c5}) != 5'd0) && r6)
|| (((line[31:35] & {c1,c2,c3,c4,c5}) != 5'd0) && r7);
      endmodule
```

5. Random

All the games should include some randomness. Hence, we researched random algorithms. We decided on the following [1]:

$$X_{n+1} = (aX_n + b) mod m$$

However, the sequence fall into repetition when any previous entity comes again. That is, the maximum period is m. To change this situation, *b* is counted up between 0 and 7 so that it can be possible that the same seed results in a different number. Then, modulus operation realized as taking last 7 bits which makes $m\ 2^7$ . *a* is adjusted to a large number filling 32 bit integer, 3215478521. Then, to make sure the repetition is prevented, three successive blocks with the same horizontal position is sensed with *change1* and *change2* parameters in the module 'Game'. If any of them comes as high, the seed of the algorithm is changed in a way to stop the repetition. That is, if the repetition takes place at right, the action tries to make the next block come from left, and vice versa.

7 bit random number will be used in the module 'Behavioralblock', and the purpose of the each bit will be explained in detail in the section of the module.

```
module random (trigger, change1, change2, out);
input trigger, change1, change2;
output[6:0] out;
reg[6:0] seed=7'd1;
integer holder=0;
reg[2:0] count=2'd0;
wire change1, change2;
assign out=seed;
always @ (posedge trigger)
begin
if (change1) seed=1010101; // If 3 successive blocks at left, turn to right
if (change2) seed=0101010; // If 3 successive blocks at right, turn to left
if (!change1 && !change2)
begin
holder = 3215478521*seed+count; // X(n+1)=a*X(n)+b
seed = holder[6:0]; // modulus operation
if(count==7) count = 0;
else count = count+1;
end
end
      endmodule
```

6. Rotatedot

The purpose of this module is only make the latest pressed button effective while two dot-controlling buttons pressed together. It is thought that it will be beneficial for more fluent gameplay. It is done by comparing buttons' previous positions with current positions.

Debouncing is not implemented for two dot-controlling buttons because their purpose is continuous action.

```
module rotatedot (frame, ccwbutton, cwbutton, ccw, cw);
input frame, ccwbutton, cwbutton;
output ccw, cw;
reg ccwold=0, cwold=0, ccw=0, cw=0;
always @ (posedge frame)
begin
ccwold<=ccwbutton; // The previous values are held
cwold<=cwbutton;
if (ccwbutton && cwbutton) begin ccw<=0; cw<=0; end
if (!ccwbutton && cwbutton) begin ccw<=1; cw<=0; end
if (ccwbutton && !cwbutton) begin ccw<=0; cw<=1; end
if (!ccwbutton && !cwbutton) // If two of them is pushed together
begin
if (ccwold && cwold) begin ccw<=0; cw<=0; end
if (!ccwold && cwold) begin ccw<=0; cw<=1; end // If CW is unpressed
previously, it is
pressed lastly
if (ccwold && !cwold) begin ccw<=1; cw<=0; end
end
end
      endmodule
```

7. Statedecider

The game has four states; start screen, gameplay, pause and end screen. This module generates the state variable taking inputs start button, pause button and collision informer variable. The ASMD chart is provided at figure 7. Almost all of the modules acts according to the state generated. Figure 3, 4, 5, 6 shows the screens at these states.


Figure 3: Start Screen

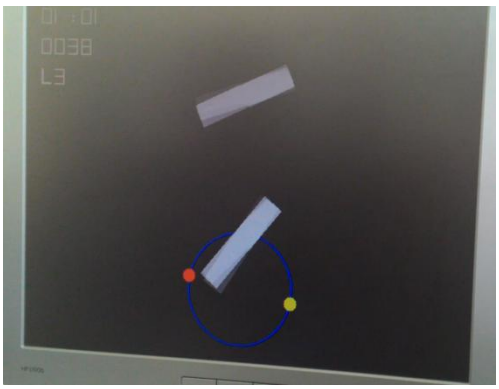

Figure 4: Game Play

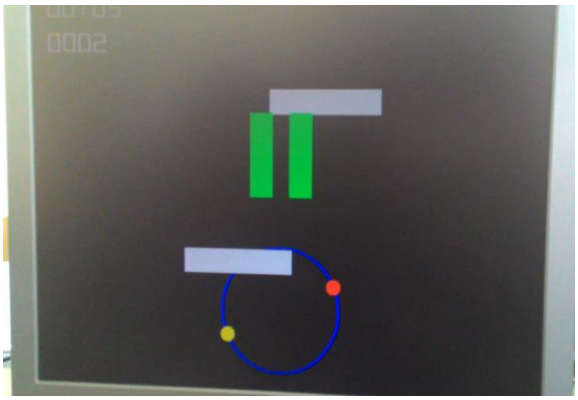


Figure 5: Pause Screen


Figure 6: End Screen

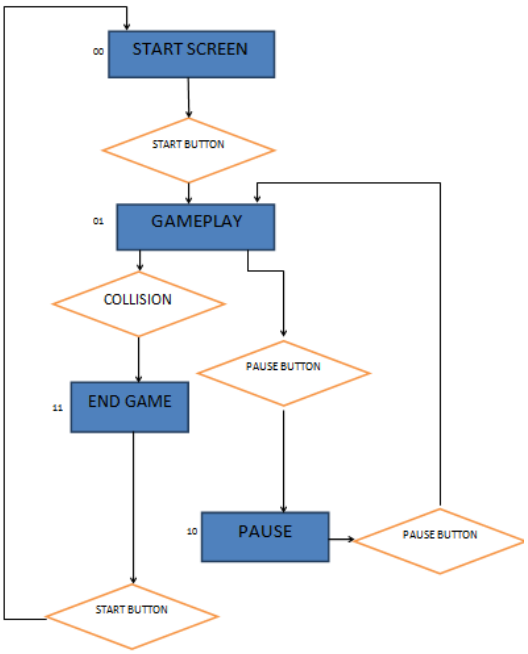


Figure 7: Flow Chart of the Game States

```
module statedecider (clk_frame, startbuttondb , pausebuttondb , collision,
state);
input clk_frame, startbuttondb , pausebuttondb , collision;
output[1:0] state;
reg[1:0] state=2'b00;

// state 00 starting screen
// state 01 game
// state 10 pause
// state 11 collision, end game

always @ (posedge clk_frame)
begin // The implementation of the flowchart
if (startbuttondb && state==2'b00) state=2'b01;
if (pausebuttondb && state==2'b01) state=2'b10;
```

```
if (collision && state==2'b01) state=2'b11;
if (pausebuttondb && state==2'b10) state=2'b01;
if (startbuttondb && state==2'b11) state=2'b00;
end
endmodule
```

8. Timecounter

This module, counts time taking advantage of the fact that the monitor completes the whole screen (a frame) nearly one sixtieth of a second. With the frame clock generated at the module 'game', a counter counts up to 60. Every cycle of this counter means a second, and four digits of time (XX:XX) are counted in 4 bit Binary Coded Decimal format. In addition, the module generates a game level named *difficulty* counting up at every 30 seconds till five.

```
module timecounter (frame, state, min1, min0, sec1, sec0, difficulty);
input frame;
input[1:0] state;
output[3:0] min1, min0, sec1, sec0, difficulty;
reg[3:0] min1=4'b0000, min0=4'b0000, sec1=4'b0000, sec0=4'b0000,
reg[3:0] difficulty=4'b0001;
reg[31:0] framecount=0;
wire frameout, sec0out, sec1out, min0out, min1out;
assign frameout = framecount==59;
assign sec0out = sec0==9;
assign sec1out = sec1==5;
assign min0out = min0==9;
assign min1out = min1==5;

// Time min1 min0 : sec1 sec0 (BCD Representation)
always @ (posedge frame)
begin
if (state==2'b01)
begin
      if (!frameout)
            framecount=framecount+1;
      else begin // If frame count is 59, frame count turns to 0
            framecount=0; // And sec0 increments ( if it's not 9)
      if(!sec0out) begin
            sec0=sec0+1;
      end
      else begin // If sec0 is 9
            sec0=0;
            if(!sec1out) begin
                  sec1=sec1+1;
            end
            else begin
                  sec1=0;
            if (!min0out) begin
                  min0=min0+1;
            end
            else begin
                  min0=0;
            if(!min1out) begin
                  min1=min1+1;
            end
```

```
            else begin
            min1=0;
            end
end
end
end
end
//Level Generation
if ((sec1==4'd3 || sec1==4'd0) && sec0==4'd0 && framecount==0 &&
difficulty!=4'd5)
difficulty=difficulty+4'd1;
end
if(state==2'b00) // If game turns to start screen, module restarts
begin
framecount=4'b0000;
min1=4'b0000;
min0=4'b0000;
sec1=4'b0000;
sec0=4'b0000;
difficulty=4'd1;
end
end
endmodule
```

9. Blockmaster

This module checks the positions of 4 obstacles and makes a new obstacle begin to fall whenever the previous obstacle reaches a certain position and whenever there is an available obstacle which has completed the fall. That is, this module enables 4 blocks appropriately. In the beginning, the first block is triggered to enable with the state change, and the process continuous as a loop. When the state changes from the end screen to start screen, the module removes the obstacles from the screen and makes them reset their parameters in the module 'Behavioralblock' for a new game.

```
module blockmaster (trigger, state,
newrequest1, newrequest2, newrequest3, newrequest4,
yend1, yend2, yend3, yend4,
enable1, enable2, enable3, enable4);
input trigger, newrequest1, newrequest2, newrequest3, newrequest4,
yend1, yend2, yend3, yend4;
input[1:0] state;
output enable1, enable2, enable3, enable4;
reg enable1=0, enable2=0, enable3=0, enable4=0, starting=1;
reg newrequest1local =0, newrequest2local =0, newrequest3local =0,
newrequest4local =0;
always @ (posedge trigger) begin
if(state==2'b01) // It is not needed but just in case
begin //(in pause or endgame there is no yend or newrequestlocal)
if (starting)
begin enable1<=1; starting<=0; end // only in the begining, it is needed to
start
if (newrequest1local && !enable2) // If there is need for new block and if
there is
begin enable2<=1; newrequest1local <=0; end // available new block, a new
block is
if (newrequest2local && !enable3) // initiated
```

```
begin enable3<=1; newrequest2local <=0; end
if (newrequest3local && !enable4)
begin enable4<=1; newrequest3local <=0; end
if (newrequest4local && !enable1)
begin enable1<=1; newrequest4local <=0; end
if (yend1 && enable1) enable1<=0;// if a block comes to the end
if (yend2 && enable2) enable2<=0;// it is removed
if (yend3 && enable3) enable3<=0;
if (yend4 && enable4) enable4<=0;
if (!newrequest1local && newrequest1 && !enable2) newrequest1local <=1; //
Purpose is to
if (!newrequest2local && newrequest2 && !enable3) newrequest2local <=1; //
initiate only
if (!newrequest3local && newrequest3 && !enable4) newrequest3local <=1; // one
block because
if (!newrequest4local && newrequest4 && !enable1) newrequest4local <=1; //
newrequest's are
// continuously high for a time
end
if(state==2'b00) // When a new game starts, removes all the blocks and
begin // leaves a trigger for the first block to start the flow again
enable1<=0; enable2<=0; enable3<=0; enable4<=0; starting<=1;
end
end
      endmodule
```

10. Behavioralblock

Main purpose of this module is creating an output representing an obstacle object, to be assessed as explained in our object displaying method. Since there is a lot of work this module does, it will explained as titles.

Random Utilization and Block Specification Deciding: The module takes 7 bit random number as input. Every random number defines a single obstacle type with its behavior, size and horizontal position specifications. Their meaning are illustrated in Table 1.

| 7 *bit number*: $r_6r_5r_4\ r_3r_2r_1r_0$ | | | | | | | | | |
|---|---|---|---|---|---|---|---|---|---|
| $r_6r_5$ | Behavior | Prob. | | $r_4\ r_3$ | Size | Prob. | | $r_2r_1r_0$ | Horizontal Pos. | Prob. |
| 00 | Simple | 1/4 | | 00 | Square | 1/4 | | 000-001 | Middle | 1/4 |
| 01 | Rotating | 1/4 | | 01 | Short | 1/4 | | 010-011-100 | Left | 3/8 |
| 10 | Horizontally Moving | 1/4 | | 10 | Long | 1/2 | | 101-110-111 | Right | 3/8 |
| 11 | Disappearing | 1/4 | | 11 | | | | | | |

Table 1: Meaning of the Random Numbers

As it is seen, the probabilities are tried to be adjusted. Blocks coming from middle should be rare, and long obstacles should be often for a better game play.

In addition, this random number should be utilized in order the prevent undesired obstacles such as rotating-short or simple-long-middle. Another purpose in utilizing is difficulty adjustment. Difficult combinations should come later in the game. The modes are enabled in the sort of simple, rotating, horizontally moving and disappearing with level. In addition, the permitted combination types are illustrated in Table 2.

| Behavior | Size | Horizontal Pos. |
|---|---|---|
| Simple & Disappearing | Square | Right, Left, Middle |
| | Short | Right, Left, Middle |
| | Long | Right, Left |
| Rotating | Long | Middle |
| Horizontally Moving | Size | Right, Left |
| | Square | Right, Left |
| | Short | Right, Left |

Table 2. Permitted Specification Types For Obstacles

The utilization can be examined further in the code. *instruction* holds the original random number and *definitionused* holds the utilized one.

Speed Adjustment: According to the level of the game held by *difficulty*, speed is incremented appropriately. For 5 levels, there is 3 speeds. Since, the levels control both the block behaviors and speed, all of 5 levels have difference although there is 3 speed levels. At every frame, vertical position of the block is increased as much as speed.

Obstacle Behavior Implementation: The horizontally moving block still takes initial horizontal position according to the last 3 bits of the random number. If it started from right, it goes left and passed through the region of dots from right, and vice versa. For better game play, this type of obstacles stays at right edge and left edge for a while during their movement.

The rotating block increments or decrements *angle* variable as much as speed, that is two times of speed in physical degrees. For a perfect passing, it is seen that the obstacle should have 90 and 270 degrees while entering to and coming out of the dot region. The angle steps and radius of the predefined circle which dots are moving on are adjusted accordingly. The angular parameterization of the obstacle will be explained at trigonometry section.

The disappearing block disappears with the help of *fade* variable generated inside the module and the operation takes place at the module 'Game' at "Color Giving Block". If the type of the block is disappearing, module outputs *fade* high when the vertical position reaches a certain value. And if the *fade* is high, the block is not colored and ignored by the "Color Giving Block". When the state is end game, the "Color Giving Block" starts to operate on the block again and the user sees the disappearing block if it is collided with.

State Reaction: The module changes the orientation and position of the obstacle only if the state is game play. This makes the pause state possible.

Interaction with the modules 'Blockmaster' and 'Scorecounter': Interaction is done with reporting signals *newrequest, score* and *yend.* All are get high when the vertical position of the block reaches certain values. *newrequest* triggers new blocks to enable, *score* increments the score and *yend* informs that the block is completed the fall and enable to a new fall.

Trigonometry: The angle of the obstacle can be adjusted along with its size and position. To do this, trigonometrical relations are used. To realize it, tangent, sine and cosine functions are generated with look up tables. While sine and cosine are generated exactly the same as in the module 'Game', for the tangent values, $f(x) = 1024 * \tan(2x)$ is used. Since the edges on the sides have $\theta$+90 degree with the horizon, there is two tangent values defined *tanblock1a* and *tanblock1b*.

$tanblock1a = 1024 * \tan(2 * angle)$,

$tanblock1a = 1024 * \tan(2 * angle + 90)$,

$sinblock1 = 1024 * (1 + \sin(2 * angle))$,

$cosblock1 = 1024 * (1 + \cos(2 * angle))$, where *angle* is the code variable.

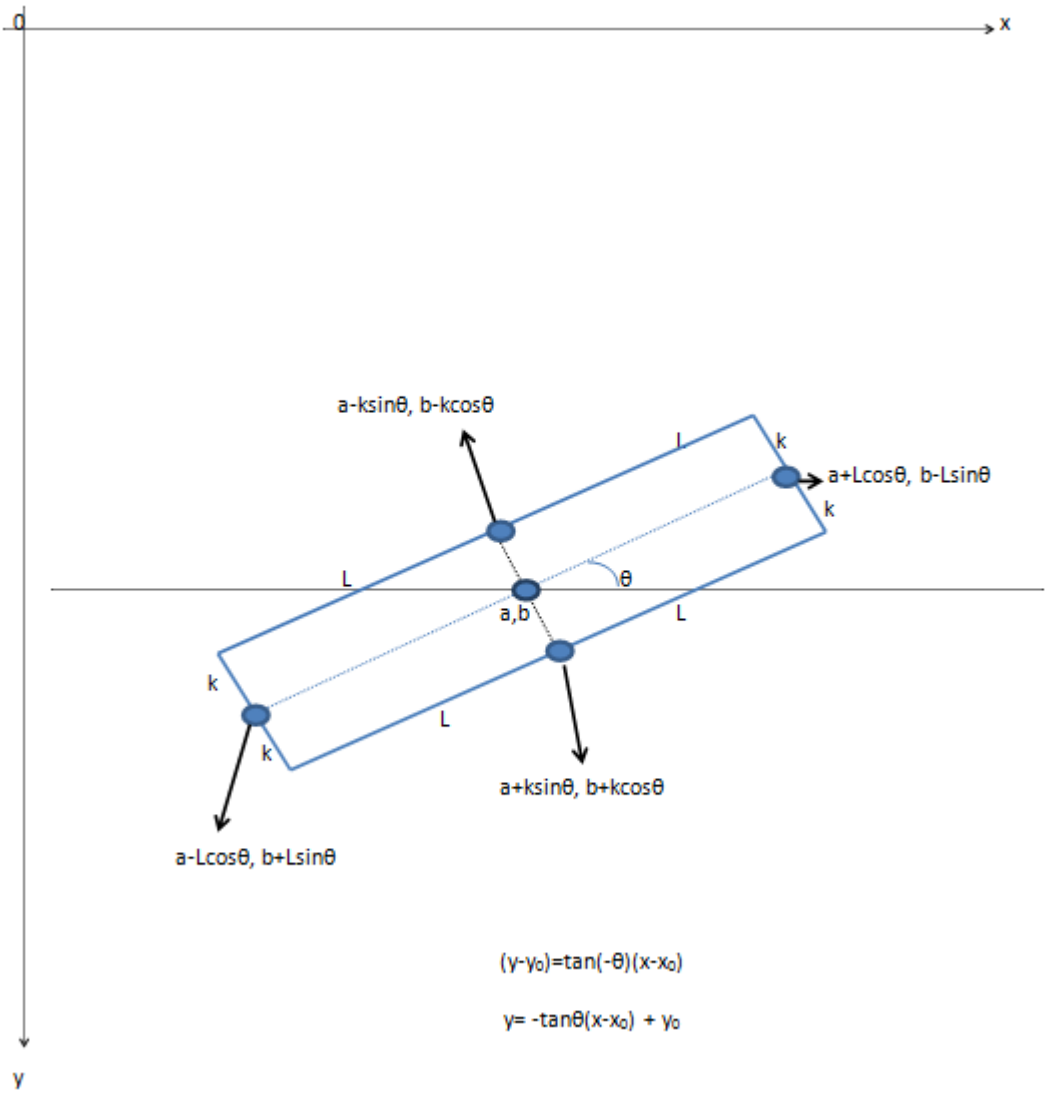


Figure 8: A Block with Nonzero Angle with the Horizon

Figure 8 shows an example obstacle with nonzero degree, center a, b and size 2k, 2L. The centers of the edges are found as in the figure. Then the horizontal and vertical positions of the 4 centers of the 4 edges are given to the equation seen under the block in the figure as $y_0$ and $x_0$. 4 line equations are obtained. In fact, four y values are obtained for one value of the x, the horizontal position of the monitoring process. During the monitoring process, position of the process x and y are compared appropriately in the light of these equations. Hence, a variable which is high when the monitor is processing inside the block and low when not is obtained. It is what we want for our object displaying method.

```
module behavioralblock (x, y, clk, enable, state, instructionin , difficulty,
posx, newrequest, yend, score, fade,out);
input[31:0] x, y;
input clk, enable;
input[1:0] state;
input[3:0] difficulty;
input[6:0] instructionin ;
output newrequest, yend, score, fade, out;
output[31:0] posx;
wire out;
integer posx=0, posy=0, mode10posx;
reg yend=0, newrequest=0, score=0, mode10init=1, mode10dir=0, mode01init=1,
fade=0;
reg[6:0] instruction=0;
reg[8:0] angle=0;
integer sinusoidal[0:45];
integer tangential[0:45];
wire[31:0] toplimity, downlimity, toplimitx, downlimitx, sinblock1, cosblock1,
wire[31:0] tanblock1a, tanblock1b, factor1, factor2, factor3, factor4,
wire[31:0] factor5, factor6, factor7, factor8, sizex, sizey;
wire[6:0] definition;
reg[6:0] definitionused =0;
wire[3:0] speed;
// Representative Variable Generating Block
assign sinblock1 = (angle<46)?sinusoidal[45-angle]:(angle>45 &&
angle<91)?sinusoidal[angle-45]
:(angle>90 && angle<136)?(2048-sinusoidal[135-angle])
:(angle>135 && angle<180)?(2048-sinusoidal[angle-135]):31'bx;
assign cosblock1 = (angle<46)?sinusoidal[angle]:(angle>45 && angle<91)?(2048-
sinusoidal[90-angle])
:(angle>90 && angle<136)?(2048-sinusoidal[angle-90])
:(angle>135 && angle<180)?sinusoidal[180-angle]:31'bx;
assign tanblock1a = (angle<46)?tangential[angle]:(angle>45 && angle<91)?(-
1*tangential[90-angle])
:(angle>90 && angle<136)?(tangential[angle-90])
:(angle>135 && angle<180)?(-1*tangential[180-angle]):31'bx;
assign tanblock1b = (angle<46)?(-1*tangential[45-angle])
:(angle>45 && angle<91)?(tangential[angle-45])
:(angle>90 && angle<136)?(-1*tangential[135-angle])
:(angle>135 && angle<180)?(tangential[angle-135]):31'bx;
assign factor1 = sizey*sinblock1;
assign factor2 = sizey*cosblock1;
assign factor3 = sizex*sinblock1;
assign factor4 = sizex*cosblock1;
assign factor5 = (((tanblock1a[31]==1)?(-1):1)*((tanblock1a[31]==1)?(-
tanblock1a):tanblock1a)
```

```
*(-x + posx -((factor1[31] ==1)?(-((-factor1) >> 10)):(factor1 >> 10)) +
sizey));
assign factor6 = ((tanblock1a[31]==1)?(-1):1)*((tanblock1a[31]==1)?(-
tanblock1a):tanblock1a)
*(-x + posx + ((factor1[31] ==1)?(-((-factor1) >> 10)):(factor1 >> 10)) -
sizey);
assign factor7 = ((tanblock1b[31]==1)?(-1):1)*((tanblock1b[31]==1)?(-
tanblock1b):tanblock1b)
*(-x + posx + ((factor4[31] ==1)?(-((-factor4) >> 10)):(factor4 >> 10)) -
sizex);
assign factor8 = ((tanblock1b[31]==1)?(-1):1)*((tanblock1b[31]==1)?(-
tanblock1b):tanblock1b)
*(-x + posx - ((factor4[31] ==1)?(-((-factor4) >> 10)):(factor4 >> 10)) +
sizex);
assign toplimity = ((factor5[31] == 1)?(-((-factor5) >> 10)):(factor5 >> 10))
+ posy
- ((factor2[31] ==1)?(-((-factor2) >> 10)):(factor2 >> 10))
+sizey;
assign downlimity = ((factor6[31] == 1)?(-((-factor6) >> 10)):(factor6 >> 10))
+ posy
+ ((factor2[31] ==1)?(-((-factor2) >> 10)):(factor2 >> 10))
- sizey;
assign toplimitx = ((factor7[31] == 1)?(-((-factor7) >> 10)):(factor7 >> 10))
+ posy
- ((factor3[31] ==1)?(-((-factor3) >> 10)):(factor3 >> 10))
+ sizex;
assign downlimitx = ((factor8[31] == 1)?(-((-factor8) >> 10)):(factor8 >> 10))
+ posy
+ ((factor3[31] ==1)?(-((-factor3) >> 10)):(factor3 >> 10))
- sizex;
assign out = enable && (angle!=0 && angle!=45 && angle!=90 && angle!=135) //
at these angles on
of the tangents
?(y>((toplimity[31]==0)?toplimity:0) ^ y>((downlimity[31]==0)?downlimity:0)
)// is infinite.
&& (y>((toplimitx[31]==0)?toplimitx:0) ^ y>((downlimitx[31]==0)?downlimitx:0)
)
: (angle==0 || angle==90) ?
( x<posx+sizex && x>posx-sizex && (y<(posy+sizey) && ((posy+sizey)>>31)==0)
&& (y>(posy-sizey) || ((posy-sizey)>>31)==1))
: (angle==45 || angle==135) ?
( x<posx+sizey && x>posx-sizey && (y<(posy+sizex) && ((posy+sizex)>>31)==0)
&& (y>(posy-sizex)|| ((posy-sizex)>>31)==1))
:0;
// End of Representative Variable Generating Block
// Random Utilization for only the Behavior of the block
assign definition[6:5]=(difficulty<2)?2'b00
:(difficulty<3 && instruction[6:5]==2'b01)?2'b01
:(difficulty<3)?2'b00
:(difficulty<4 && instruction[6:5]==2'b11)?2'b00
:instruction[6:5];
// Size Deciding
assign sizex= (definitionused [4]==1'b0)?30:70;
assign sizey= (definitionused [4:3]==2'b00)?30:15;
// Speed adjusting according to level
assign speed= (difficulty<3)?4'd2:(difficulty<5)?4'd3:4'd4;
initial begin
```

```
sinusoidal[0]=2048;
sinusoidal[1]=2047;
sinusoidal[2]=2046;
sinusoidal[3]=2042;
sinusoidal[4]=2038;
sinusoidal[5]=2032;
sinusoidal[6]=2026;
sinusoidal[7]=2018;
sinusoidal[8]=2008;
sinusoidal[9]=1998;
sinusoidal[10]=1986;
sinusoidal[11]=1973;
sinusoidal[12]=1959;
sinusoidal[13]=1944;
sinusoidal[14]=1928;
sinusoidal[15]=1911;
sinusoidal[16]=1892;
sinusoidal[17]=1873;
sinusoidal[18]=1852;
sinusoidal[19]=1831;
sinusoidal[20]=1808;
sinusoidal[21]=1785;
sinusoidal[22]=1761;
sinusoidal[23]=1735;
sinusoidal[24]=1709;
sinusoidal[25]=1682;
sinusoidal[26]=1654;
sinusoidal[27]=1626;
sinusoidal[28]=1597;
sinusoidal[29]=1567;
sinusoidal[30]=1536;
sinusoidal[31]=1505;
sinusoidal[32]=1473;
sinusoidal[33]=1440;
sinusoidal[34]=1408;
sinusoidal[35]=1374;
sinusoidal[36]=1340;
sinusoidal[37]=1306;
sinusoidal[38]=1272;
sinusoidal[39]=1237;
sinusoidal[40]=1202;
sinusoidal[41]=1167;
sinusoidal[42]=1131;
sinusoidal[43]=1095;
sinusoidal[44]=1060;
sinusoidal[45]=1024;
tangential[0]=0;
tangential[1]=36;
tangential[2]=72;
tangential[3]=108;
tangential[4]=144;
tangential[5]=181;
tangential[6]=218;
tangential[7]=255;
tangential[8]=294;
tangential[9]=333;
tangential[10]=373;
```

```
tangential[11]=414;
tangential[12]=456;
tangential[13]=499;
tangential[14]=544;
tangential[15]=591;
tangential[16]=640;
tangential[17]=691;
tangential[18]=744;
tangential[19]=800;
tangential[20]=859;
tangential[21]=922;
tangential[22]=989;
tangential[23]=1060;
tangential[24]=1137;
tangential[25]=1220;
tangential[26]=1311;
tangential[27]=1409;
tangential[28]=1518;
tangential[29]=1639;
tangential[30]=1774;
tangential[31]=1926;
tangential[32]=2100;
tangential[33]=2300;
tangential[34]=2534;
tangential[35]=2813;
tangential[36]=3152;
tangential[37]=3571;
tangential[38]=4107;
tangential[39]=4818;
tangential[40]=5807;
tangential[41]=7286;
tangential[42]=9743;
tangential[43]=14644;
tangential[44]=29324;
tangential[45]=32'dx;
end
always @ (posedge clk)
begin
// Random Utilization For Size and Horizontal Position
definitionused [4:0]=(((definitionused [6:5]==2'b00 || definitionused
[6:5]==2'b11)
&& instruction[4]==1'b1) || definitionused [6:5]==2'b10)
?({instruction[4:3],2'b10,instruction[0]})
:((definitionused [6:5]==2'b00 || definitionused [6:5]==2'b11) &&
instruction[4]==1'b0)
?instruction[4:0]
:5'b10000;
if (enable && !yend)
begin
// This block increments or decrements the parameters of the block.
if(state==2'b01) // These operations only done in gameplay state. If State is
pause
begin // it neither does these operations nor resets the parameters.
if(definitionused [6:5]!=2'b10) // The horizontal allignment for all except
begin // the horizontally moving type
if(definitionused [2:1]==2'b00) posx<=320;
```

```
else if(definitionused [2:1]==2'b01 || definitionused [2:0]==3'b100)
posx<=265;
else posx<=375;
end
else // Horizontally moving block
begin
if (mode10init)
begin
if(definitionused [0]) begin mode10posx<=241; mode10dir<=1; mode10init<=0; end
else begin mode10posx<=399; mode10dir<=0; mode10init<=0; end
end
if (!mode10init && mode10dir)
begin
if (mode10posx>(409-speed)) mode10dir<=0;
mode10posx<=mode10posx+speed;
end
if (!mode10init && !mode10dir)
begin
if (mode10posx<(231+speed)) mode10dir<=1;
mode10posx<=mode10posx-speed;
end
if (mode10posx>375) posx<=375;
else if (mode10posx<265) posx<=265;
else posx<=mode10posx;
end
if(definitionused [6:5]==2'b01) // Rotating mode
begin
if(mode01init) begin angle<=156; mode01init=0; end
else begin
if(angle>(179-speed)) angle <= (angle+speed-180);
else angle<=angle+speed;
end
end
else angle<=0;
posy<=posy+speed; // Vertical movement
if (posy>180) newrequest<=1; // Interactions with outside
if (posy>250 && definitionused [6:5]==2'b11) fade<=1;
if (posy>470) score<=1;
if (posy>520) yend<=1;
end
end
else begin
posx<=0; posy<=0;
end
if (!enable) // enable variable controlled by 'Blockmaster' module.
begin // When it is zero, everything resets.
instruction<=instructionin ;
definitionused [6:5]<=definition[6:5];
yend<=0;
newrequest<=0;
fade<=0;
score<=0;
posx<=0;
posy<=0;
mode10init<=1;
mode01init<=1;
angle<=0;
```

```
end
end
      endmodule
```

11. Scorecounter

This module counts the number of successively passed obstacles. The logic is the same with the module 'Timecounter'. However, the triggering signals *score1, score2, score3, score4* coming from 'Behavioralblock' modules stays high for a period of time. To prevent multiple increments, there is some manipulations.

```
module scorecounter (frame, state, score1, score2, score3, score4, s3, s2, s1,
s0);
input frame, score1, score2, score3, score4;
input[1:0] state;
output[3:0] s3, s2, s1, s0;
reg[3:0] s3=0, s2=0, s1=0, s0=0;
reg score1done=0, score2done=0, score3done=0, score4done=0;
wire s0out, s1out, s2out, s3out;
assign s0out = s0==9;
assign s1out = s1==9;
assign s2out = s2==9;
assign s3out = s3==9;
// Score; s3 s2 s1 s0 (BCD representation)
always @ (posedge frame)
begin
if (state==2'b01 && ((score1 && !score1done) || (score2 && !score2done)
|| (score3 && !score3done) || (score4 && !score4done)))
begin
if(~s0out) s0<=s0+1; // If s0 is 9 it turns to 0 and increments s1
else begin
s0=0;
if(~s1out) s1<=s1+1;
else begin
s1=0;
if (~s2out) s2<=s2+1;
else begin
s2=0;
if(~s3out) s3<=s3+1;
else begin
s3<=0;
end
end
end
end
if (score1 && !score1done) score1done<=1; // scorexdone aims not to
if (score2 && !score2done) score2done<=1; // increment continuously because
scorex
if (score3 && !score3done) score3done<=1; // stays high for a period of time
if (score4 && !score4done) score4done<=1; // It should increment a single time
end
if(!score1) score1done<=0;
if(!score2) score2done<=0;
if(!score3) score3done<=0;
if(!score4) score4done<=0;
if (state==2'b00) // Resets while turning to Start screen
```

```
begin
s3<=0;
s2<=0;
s1<=0;
s0<=0;
score1done<=0;
score2done<=0;
score3done<=0;
score4done<=0;
end
end
      endmodule
```

12. Highscore

The module holds the highest value. It compares current score and the one it holds. If current score is greater, it sets the value it holds to the current value. If this action happens at least one time in a game, it gives the *highhappened* output as high. According to this output, high score information text is shown or not at the end screen.

```
module highscore (clk_frame,s3,s2,s1,s0,shigh3,shigh2,shigh1,shigh0,
highhappened );
input clk_frame;
input[3:0] s3,s2,s1,s0;
output[3:0] shigh3,shigh2,shigh1,shigh0;
output highhappened ;
reg[3:0] shigh3=4'd0,shigh2=4'd0,shigh1=4'd0,shigh0=4'd0;
wire act;
reg highhappened =0;
assign act=({s3,s2,s1,s0}>{shigh3,shigh2,shigh1,shigh0});// Whether the
current score
// is greater or not
always @ (posedge clk_frame)
begin
if(act) // if high score
begin
highhappened =1'b1; // according to this, there will be "High Score!" text or
not
shigh3=s3; // The high score held
shigh2=s2;
shigh1=s1;
shigh0=s0;
end
if (s3==0 && s2==0 && s1==0 && s0==0) highhappened =1'b0; // restart for the
new game
// the condition satisfies only
// the state turns to start screen
end
      endmodule
```

The project is implemented on Altera De1 FPGA an on a monitor with VGA interface. The *r, g, b* outputs of the main module are assigned the most significant bits of the DAC governing R, G and B pins of the VGA interface. hs, vs, and clk_screen are assigned to related pins of the VGA interface. The figure 9 shows assignments done.

| | tatu | From | To | Assignment Name | Value | Enabled | Entity | Comment | Tag |
|---|---|---|---|---|---|---|---|---|---|
| 1 | ✓ | | out hs | Location | PIN_B11 | Yes | | | |
| 2 | ✓ | | out vs | Location | PIN_D11 | Yes | | | |
| 3 | ✓ | | out r | Location | PIN_F13 | Yes | | | |
| 4 | ✓ | | out g | Location | PIN_E11 | Yes | | | |
| 5 | ✓ | | out b | Location | PIN_J14 | Yes | | | |
| 6 | ✓ | | out clk_screen | Location | PIN_A11 | Yes | | | |
| 7 | ✓ | | in clk | Location | PIN_AF14 | Yes | | | |
| 8 | ✓ | | in pausebutton | Location | PIN_AA15 | Yes | | | |
| 9 | ✓ | | in startbutton | Location | PIN_AA14 | Yes | | | |
| 10 | ✓ | | in ccwbutton | Location | PIN_Y16 | Yes | | | |
| 11 | ✓ | | in cwbutton | Location | PIN_W15 | Yes | | | |
| 12 | | <<new>> | <<new>> | <<new>> | | | | | |

Figure 9: Pin Assignments

## TEST

Testing process of the project is conducted on FPGA supplied with by the department whenever applicable. For working at home, a simulation program VGASim is used.

## CONCLUSION

After a work conducted for one and a half month, we got to use Verilog HDL and FPGA. A lot of research was conducted on online sources. We learned how to think while programming with a HDL. It requires knowledge on sequential and combinational logic circuits. We learned how VGA interface works and how to use it for obtaining desired display.

We work on algorithm more than visual quality. We could have used all of the bits of DAC's governing color bits of VGA to obtain more colors and use different ready to use programs to obtain visuals with more quality. Instead, we always think simply and generate our ideas to realize our aims about the project. To realize rotating blocks were our aim in the beginning. Then, a number of different aspects were added such as disappearing blocks, horizontally moving blocks, random flow and so on. We think, it was wise to use representative one bit variables for displaying objects rather than using a memory matrix for all the pixels in the screen. We realize the 7 segment display we used in the laboratory experiments for displaying digits, which leads us to use 35 segment display for letters later.

At last, we are happy to see that our project becomes a game which is played with joy.

## BIOGRAPHY

I am Kaan Menekşedağ and I was born in 1993, Bursa. I study in Electrical and Electronic Engineering in Middle East Technical University and I graduated from Tan

Science High School, Bursa. I was always interested in mathematics and physics. I am a board member of Biomedical Student Society since 2013.

My name is Anıl Kurt. I was born in 1993, Antalya. I am a student in Electrical and Electronics Engineering Department in METU. I graduated from Antalya Anatolian High School. In METU, Best Term Project certificate is awarded to me and my partners by EE230 Course Committee at June 2014.

REFERENCE


[1] http://en.wikipedia.org/wiki/Random_number_generation, retrieved 07.06.2015

[2] https://odtuclass.metu.edu.tr/pluginfile.php/112757/mod_resource/content/2/EE314-Projects-Spring2014_2015%20%28v3%29.pdf, retrieved 07.06.2015

[3] https://odtuclass.metu.edu.tr/pluginfile.php/114531/mod_resource/content/1/General%20Rules% 20for%20EE314%20Projects.pdf, retrieved 07.06.2015

[4] https://inst.eecs.berkeley.edu/~cs150/fa06/Labs/verilog-ieee.pdf, retrieved 07.06.2015


APPENDIX

```
module game (clk, pausebutton, startbutton, ccwbutton, cwbutton, hs, vs, r, g, b, clk_screen);
input clk; // 50 MHz internal clock of the FPGA
input pausebutton, startbutton, ccwbutton, cwbutton; // buttons (active low)
output hs, vs, r, g, b, clk_screen; // VGA outputs
reg clk_screen=1'b0, clk_frame=1'b0; // 25MHz and (nearly) 60Hz generated clocks
reg pausebuttondb =1'b0, startbuttondb =1'b0; // debounced and active high buttons
reg collision=1'b0, hs=1'b0, vs=1'b0;
reg[2:0] pausecounter =3'b000, startcounter =3'b000; // used for debouncing
reg[2:0] color=3'b000; // its bits will be connected to r, g, b outputs respectively
reg [8:0] dotangle=8'd0; //
reg[10:0] xcount=10'd0; //holds horizontal VGA process(screen(640)
//+forthporch(16)+hs(96)+backporch(48))
reg[10:0] ycount=10'd1; //holds vertical VGA
process(screen(480)+forthporch(10)+vs(2)+backporch(33))
```

```
integer sinusoidal[0:45];// nth entry corresponds to 1024*(1+cos(2n)) for
n=0..45
wire r, g, b; // VGA outputs
wire backcircle, dot1, dot2, block1, block2,
wire block3, block4, pauseindicator ; // This block of wires all
wire char1, char2, charsc, char3, char4,
wire char5, char6, char7, char8, char9; // represent different visual
wire char10, char11, char12, go_g, go_a,
wire go_m, go_e, go_o, go_v, go_e2, go_r; // objects. When the monitor
wire dipole_d, dipole_i, dipole_p,
wire dipole_o, dipole_l, dipole_e; // comes to the pixels that the
wire high_h, high_i, high_g, high_h2,
wire high_sc, lev_l, lev_s; // object is desired to take place;
wire high2_h, high2_i, high2_g, high2_h2,
wire high2_s, high2_c, high2_o, high2_r; // the wire becomes '1', otherwise
'0'.
wire high2_e, high2_donk; // Later, we give rgb a value
wire score_s, score_c, score_o, score_r, score_e, score_sc;// accordingly in
order to give a
wire time_t ,time_i ,time_m ,time_e ,time_sc; // color the object.
wire newrequest1, newrequest2, newrequest3,
wire newrequest4, yend1, yend2; // connections between modules
wire yend3, yend4, enable1, enable2, enable3,
wire enable4, score1, score2; // "behavioralblock" and "blockmaster"
wire score3, score4
wire fade1, fade2, fade3, fade4; // output of "behavioralblock". Block
dissappears accordingly
wire highhappened ; //in this module output of "highscore". Whether a game
ended with a high score.
wire cw, ccw; //At the end game screen, high score is informed if it's '1'.
wire change1, change2; //output of "rotatedot". Dots rotate accordingly.
//Whether 3 successive blocks have same horizontal position.
//Random seed is changed accordingly.
wire[3:0] min1, min0, sec1, sec0, s3, s2, s1, s0, shigh3; // Some outputs
wire[3:0] shigh2, shigh1, shigh0, difficulty, speed; // and inputs of modules
wire[6:0] randomnumber ; // and connections
wire[31:0] scorex3,scorex2,scorex1,scorex0,scorey; // in between. All
wire[31:0] timex3, timex2, timesc, timex1, timex0, timey; // will be
wire[31:0] posxblock1, posxblock2, posxblock3, posxblock4; // explained
wire[1:0] state; // describes 4 states of the game (detail : state module)
wire[31:0] x, y; // holds the position of the visible monitoring process
// (screen portion of the xcount and ycount)
wire[31:0] sindot, cosdot; // sindot gives 1024*(1+sin(2*dotangle)).
// It is defined on 360 degree in 2 degree steps
assign change1= (posxblock1 == posxblock2) && (posxblock1 == posxblock3)
&& (posxblock1 == 320) && (posxblock1 == 265);
assign change2= (posxblock1 == posxblock2) && (posxblock1 == posxblock3)
&& (posxblock1 == 375);
// If 3 blocks have same horizontal position, one of them give '1'.
//Seed of the random algorithm changes accordingly.
assign sindot = (dotangle<46)?sinusoidal[45-dotangle]
:(dotangle>45 && dotangle<91)?sinusoidal[dotangle-45]
:(dotangle>90 && dotangle<136)?(2048-sinusoidal[135-dotangle])
:(dotangle>135 && dotangle<180)?(2048-sinusoidal[dotangle-135]):31'bx;
assign cosdot = (dotangle<46)?sinusoidal[dotangle]:(dotangle>45 &&
dotangle<91)
?(2048-sinusoidal[90-dotangle])
```

```
:(dotangle>90 && dotangle<136)?(2048-sinusoidal[dotangle-90])
:(dotangle>135 && dotangle<180)?sinusoidal[180-dotangle]:31'bx;
// for example;sindot gives 1024*(1+sin(2*dotangle)).It is defined on 360
degree in 2 degree steps
assign x=(xcount<640)?(xcount+1):((xcount==(800))?1:0);
assign y=(ycount>35 && ycount<516 && xcount<(800))?(ycount-35)
:(ycount>34 && ycount<515 && xcount==(800))?(ycount-34):0;
// they hold the position of the visible monitoring process
//(screen portion of the xcount and ycount).All visual object placements will
use these parameters
assign fon = (x>200)&&(x<440)&&(y>80)&&(y<155); // the yellow portion at the
back of
//the DIPOLE text at start screen
assign pauseindicator = ((x<310 && x>280)
|| (x<360 && x>330)) && (y<250 && y>150); // the universal pause indicator
seen
assign backcircle = ((x-320)**2 + (y-384)**2)<(76**2+1)//at pause screen
&& ((x-320)**2 + (y-384)**2)>(74**2-1); // Predefined ring under dots.The
geometrical ring
// equation for a circle with center
//(320,384) and 74<=r<=76
assign dot1 = ((x-(75*cosdot >> 10)-245)**2 + (y+(75*sindot >> 10)-
459)**2)<(9**2+1);
// equation for a circle with center (320+75cosx,384-75sinx) and r<=9
assign dot2 = ((x+(75*cosdot >> 10)-395)**2 + (y-(75*sindot >> 10)-
309)**2)<(9**2+1);
// equation for a circle with center (320-75cosx,384+75sinx) and r<=9
assign r=color[2]; // In order to give
assign g=color[1]; // colors on a single
assign b=color[0]; // variable 'color'
assign scorex3 = (state==2'b11)?340:20; // state 11 is end game.
assign scorex2 = (state==2'b11)?360:40; // This block aims to
assign scorex1 = (state==2'b11)?380:60; // bring "Score Panel" and
assign scorex0 = (state==2'b11)?400:80; // "Time Panel" to somewhere in middle
assign scorey = (state==2'b11)?250:60; // at the end of the game.
assign timex3 = (state==2'b11)?340:20; // At gameplay and pause states
assign timex2 = (state==2'b11)?360:40; // which are denoted with 01 and 10;
assign timesc = (state==2'b11)?380:60; // they are at top-left of
assign timex1 = (state==2'b11)?400:80; // the screen. At starting screen,
assign timex0 = (state==2'b11)?420:100; // they are not given color at "Color
Giving Block" below;
assign timey = (state==2'b11)?280:20; // hence they are not seen.
assign speed= (difficulty<3)?4'd2:(difficulty<5)?4'd3:4'd4;
// speed will be increment in y position and change in angles of dots and
blocks while rotating.
character u0 (min1,x,y,timex3,timey,char1); // Time Panel
character u1 (min0,x,y,timex2,timey,char2); //
letters usc (5'd26,x,y,timesc,timey,charsc); //
character u2 (sec1,x,y,timex1,timey,char3); //
character u3 (sec0,x,y,timex0,timey,char4); //
character sc3 (s3,x,y,scorex3,scorey,char8); // Score Panel
character sc2 (s2,x,y,scorex2,scorey,char7); //
character sc1 (s1,x,y,scorex1,scorey,char6); //
character sc0 (s0,x,y,scorex0,scorey,char5); //
letters lev1 (5'd11,x,y,20,100,lev_l); // Level Panel
character lev2 (difficulty,x,y,40,100,lev_s); //
letter lend1 (5'd6,x,y,255,70,go_g); // "Game Over" text
```

```
letter lend2 (5'd0,x,y,290,70,go_a); //
letter lend3 (5'd12,x,y,325,70,go_m); //
letter lend4 (5'd4,x,y,360,70,go_e); //
letter lend5 (5'd14,x,y,255,120,go_o); //
letter lend6 (5'd21,x,y,290,120,go_v); //
letter lend7 (5'd4,x,y,325,120,go_e2); //
letter lend8 (5'd17,x,y,360,120,go_r); //
letters lend9 (5'd7,x,y,210,200,high2_h); // "High Score!" Text
letters lend10 (5'd8,x,y,230,200,high2_i); // which appears only if
letters lend11 (5'd6,x,y,250,200,high2_g); // the game ended with
letters lend12 (5'd7,x,y,270,200,high2_h2); // a high score. Whether it
letters lend13 (5'd18,x,y,310,200,high2_s); // appears or not is
letters lend14 (5'd2,x,y,330,200,high2_c); // decided at "Color Giving
letters lend15 (5'd14,x,y,350,200,high2_o); // Block" below
letters lend16 (5'd17,x,y,370,200,high2_r); //
letters lend17 (5'd4,x,y,390,200,high2_e); //
letters lend18 (5'd27,x,y,410,200,high2_donk);//
letters lend19 (5'd18,x,y,200,250,score_s); // "Score:" Text
letters lend20 (5'd2,x,y,220,250,score_c); // at the end screen.
letters lend21 (5'd14,x,y,240,250,score_o); // "Score Panel" will come near it
letters lend22 (5'd17,x,y,260,250,score_r); // as adjusted above
letters lend23 (5'd4,x,y,280,250,score_e); //
letters lend24 (5'd26,x,y,310,250,score_sc); //
letters lend25 (5'd19,x,y,220,280,time_t); // "Time:" Text
letters lend26 (5'd8,x,y,240,280,time_i); // at the end screen.
letters lend27 (5'd12,x,y,260,280,time_m); // "Time Panel" will come near it
letters lend28 (5'd4,x,y,280,280,time_e); // as adjusted above
letters lend29 (5'd26,x,y,310,280,time_sc); //
letter lstart1 (5'd3,x,y,235,100,dipole_d); // "Dipole" Text
letter lstart2 (5'd8,x,y,265,100,dipole_i); //
letter lstart3 (5'd15,x,y,295,100,dipole_p); //
letter lstart4 (5'd14,x,y,325,100,dipole_o); //
letter lstart5 (5'd11,x,y,355,100,dipole_l); //
letter lstart6 (5'd4,x,y,385,100,dipole_e); //
letters lstart7 (5'd7,x,y,235,180,high_h); // "High:" Text
letters lstart8 (5'd8,x,y,255,180,high_i); //
letters lstart9 (5'd6,x,y,275,180,high_g); //
letters lstart10 (5'd7,x,y,295,180,high_h2); //
letters lstart11 (5'd26,x,y,312,180,high_sc); //
character hsc0 (shigh0,x,y,390,180,char9); // High Score Panel
character hsc1 (shigh1,x,y,370,180,char10); // near "High:" Text
character hsc2 (shigh2,x,y,350,180,char11); //
character hsc3 (shigh3,x,y,330,180,char12); //
// Other modules. Detailed information will be given inside every module.
timecounter t0 (clk_frame, state, min1, min0, sec1, sec0, difficulty);
scorecounter sc (clk_frame, state, score1, score2, score3, score4, s3, s2, s1,
s0);
highscore hsc (clk_frame,s3,s2,s1,s0,shigh3,shigh2,shigh1,shigh0, highhappened
);
random r0 (clk_frame, change1, change2, randomnumber );
blockmaster bm0 (clk_frame, state, newrequest1, newrequest2, newrequest3,
newrequest4,
yend1, yend2, yend3, yend4, enable1, enable2, enable3, enable4);
behavioralblock b1 (x, y, clk_frame, enable1, state, randomnumber ,
difficulty,
posxblock1, newrequest1, yend1, score1, fade1, block1);
```

```
behavioralblock b2 (x, y, clk_frame, enable2, state, randomnumber ,
difficulty,
posxblock2, newrequest2, yend2, score2, fade2, block2);
behavioralblock b3 (x, y, clk_frame, enable3, state, randomnumber ,
difficulty,
posxblock3, newrequest3, yend3, score3, fade3, block3);
behavioralblock b4 (x, y, clk_frame, enable4, state, randomnumber ,
difficulty,
posxblock4, newrequest4, yend4, score4, fade4, block4);
rotatedot ro (clk_frame,ccwbutton, cwbutton, ccw, cw);
statedecider sd (clk_frame, startbuttondb , pausebuttondb , collision, state);
initial begin
sinusoidal[0]=2048;
sinusoidal[1]=2047;
sinusoidal[2]=2046;
sinusoidal[3]=2042;
sinusoidal[4]=2038;
sinusoidal[5]=2032;
sinusoidal[6]=2026;
sinusoidal[7]=2018;
sinusoidal[8]=2008;
sinusoidal[9]=1998;
sinusoidal[10]=1986;
sinusoidal[11]=1973;
sinusoidal[12]=1959;
sinusoidal[13]=1944;
sinusoidal[14]=1928;
sinusoidal[15]=1911;
sinusoidal[16]=1892;
sinusoidal[17]=1873;
sinusoidal[18]=1852;
sinusoidal[19]=1831;
sinusoidal[20]=1808;
sinusoidal[21]=1785;
sinusoidal[22]=1761;
sinusoidal[23]=1735;
sinusoidal[24]=1709;
sinusoidal[25]=1682;
sinusoidal[26]=1654;
sinusoidal[27]=1626;
sinusoidal[28]=1597;
sinusoidal[29]=1567;
sinusoidal[30]=1536;
sinusoidal[31]=1505;
sinusoidal[32]=1473;
sinusoidal[33]=1440;
sinusoidal[34]=1408;
sinusoidal[35]=1374;
sinusoidal[36]=1340;
sinusoidal[37]=1306;
sinusoidal[38]=1272;
sinusoidal[39]=1237;
sinusoidal[40]=1202;
sinusoidal[41]=1167;
sinusoidal[42]=1131;
sinusoidal[43]=1095;
sinusoidal[44]=1060;
```

```
sinusoidal[45]=1024;
end
always @ (posedge clk) // 25 MHz clock generation. clk_screen is 25MHz, clk is
50 Mhz
begin
clk_screen=!clk_screen;
end
always @ (posedge clk_screen) // Consists of "VGA Synch Block", "Color Giving
Block"
begin // and "Collision Informer Block".Note that this portion
//operates with 25MHz clock. That is, the "VGA Synch Block"
xcount=xcount+1;
if (xcount>(800))
begin
xcount=1;
ycount=ycount+1;
end
if (ycount>525)
begin
ycount=1;
clk_frame=1;
end
else clk_frame=0;
if (xcount<657 || xcount>752) hs=1;
else hs=0;
if (ycount<3) vs=0;
else vs=1;
// "Color Giving Block"
if (x && y)
begin
if (state==2'b00) // At Start Screen
color=(high_h | high_i | high_g | high_h2 | high_sc | char9 | char10 | char11
| char12)?3'b001
:(dipole_d | dipole_i | dipole_p | dipole_o | dipole_l | dipole_e)?3'b001
:(fon)?3'b110
:(dot1)?3'b100:(dot2)?3'b110
:(backcircle)?3'b001:3'b000;
if (state==2'b01 || state==2'b10) // At Pause and Gameplay Screen
color=(pauseindicator && state==2'b10)?3'b010
:(char1 || char2 || charsc || char3 || char4 || char5 || char6 || char7 ||
char8
|| lev_l || lev_s)?3'b111
:(dot1)?3'b100:(dot2)?3'b110
:((block1 && (state==2'b11 || !fade1))| (block2 && (state==2'b11 || !fade2))
| (block3 && (state==2'b11 || !fade3)) | (block4 && (state==2'b11 ||
!fade4)))?3'b111
:(backcircle)?3'b001:3'b000;
if (state==2'b11) // At End Screen
color=(go_g | go_a | go_m | go_e | go_o | go_v | go_e2 | go_r)?3'b100
:(highhappened && (high2_h | high2_i | high2_g | high2_h2 | high2_s | high2_c
| high2_o | high2_r | high2_e | high2_donk))?3'b010
:(score_s | score_c | score_o | score_r | score_e | score_sc | time_t | time_i
| time_m | time_e | time_sc
| char1 | char2 | charsc | char3 | char4 | char5 | char6 | char7 |
char8)?3'b100
:(dot1)?3'b100:(dot2)?3'b110
:((block1 && (state==2'b11 || !fade1))| (block2 && (state==2'b11 || !fade2))
```

```
| (block3 && (state==2'b11 || !fade3)) | (block4 && (state==2'b11 ||
!fade4)))?3'b111
:(backcircle)?3'b001:3'b000;
end
else color=3'b000;
// "Collision Detector Block"
if ((dot1 || dot2) && (block1 || block2 || block3 || block4)) collision=1;
if (state==2'b00 && collision==1) collision=0;
end
always @ (posedge clk_frame)
begin
// "Dot Rotation"
if (ccw && state==2'b01)
begin
if (dotangle>(179-speed)) dotangle=dotangle+speed-180;
else dotangle=dotangle+speed;
end
if (cw && state==2'b01)
begin
if (dotangle<speed) dotangle=dotangle+180-speed;
else dotangle=dotangle-speed;
end
if (state==2'b00) dotangle=0;
// "Pause Button Debouncer"
if (!pausebutton && pausecounter ==3'b000)
begin
pausebuttondb <=1'b1;
pausecounter <=3'b001;
end
if (pausecounter !=3'b000)
begin
pausebuttondb <=1'b0;
if(pausebutton)
begin
if (pausecounter ==3'b111) pausecounter <=3'b000;
else pausecounter <=pausecounter +3'b001;
end
else pausecounter <=3'b001;
end
// "Start Button Debouncer"
if (!startbutton && startcounter ==3'b000)
begin
startbuttondb <=1'b1;
startcounter <=3'b001;
end
if (startcounter !=3'b000)
begin
startbuttondb <=1'b0;
if(startbutton)
begin
if (startcounter ==3'b111) startcounter <=3'b000;
else startcounter <=startcounter +3'b001;
end
else startcounter <=3'b001;
end
end
endmodule
```

```
module character (in, x, y, posx, posy, out);
input[3:0] in;
input[31:0] x, y, posx, posy;
output out;
wire[31:0] xcell, ycell;
wire[1:7] line;
reg[1:7] tablo[0:9];
wire a,b,c,d,e,f,g;
initial begin
tablo[0]=7'b1111110; // defining digits in terms of segments
tablo[1]=7'b0000110;
tablo[2]=7'b1101101;
tablo[3]=7'b1111001;
tablo[4]=7'b0110011;
tablo[5]=7'b1011011;
tablo[6]=7'b1011111;
tablo[7]=7'b1110000;
tablo[8]=7'b1111111;
tablo[9]=7'b1111011;
end
assign line=tablo[in];
// inner positions for only the area the module covers
assign xcell = (x>posx && x<(posx+16) && y>posy && y<(posy+22))?(x-posx):0;
assign ycell = (x>posx && x<(posx+16) && y>posy && y<(posy+22))?(y-posy):0;
assign a = (ycell>0 && ycell<3)?1:0; // segment definitions
assign b = (xcell>13 && xcell<16 && ycell<11 && ycell>0)?1:0;
assign c = (xcell>13 && xcell<16 && ycell>10 && ycell<22)?1:0;
assign d = (ycell>19 && ycell<22)?1:0;
assign e = (xcell>0 && xcell<3 && ycell>10 && ycell<22)?1:0;
assign f = (xcell>0 && xcell<3 && ycell<11 && ycell>0)?1:0;
assign g = (ycell>9 && ycell<12)?1:0;
assign out = a&(line[1]) | b&(line[2]) | c&(line[3]) | d&(line[4])
| e&(line[5]) | f&(line[6]) | g&(line[7]); // representative one bit variable
endmodule


module letter (in, x, y, posx, posy, out);
input[4:0] in;
input[31:0] x, y, posx, posy;
output out;
wire[31:0] xcell, ycell;
wire[1:35] line;
reg[1:35] tablo[0:27];
wire r1,r2,r3,r4,r5,r6,r7,c1,c2,c3,c4,c5;
initial begin
tablo[0]=35'b01110_10001_10001_11111_10001_10001_10001 ; // A
tablo[1]=35'b11110_10001_10001_11110_10001_10001_11110 ; // B
tablo[2]=35'b01111_10000_10000_10000_10000_10000_01111 ; // C
tablo[3]=35'b11110_10001_10001_10001_10001_10001_11110 ; // D
tablo[4]=35'b11111_10000_10000_11110_10000_10000_11111 ; // E
tablo[5]=35'b11111_10000_10000_11110_10000_10000_10000 ; // F
tablo[6]=35'b01111_10000_10000_10111_10001_10001_01110 ; // G
tablo[7]=35'b10001_10001_10001_11111_10001_10001_10001 ; // H
tablo[8]=35'b11111_00100_00100_00100_00100_00100_11111 ; // I
```

```
tablo[9]=35'b00010_00010_00010_00010_10010_10010_01100 ; // J
tablo[10]=35'b10001_10010_10100_11000_10100_10010_10001 ; // K
tablo[11]=35'b10000_10000_10000_10000_10000_10000_11111 ; // L
tablo[12]=35'b10001_11011_10101_10101_10001_10001_10001 ; // M
tablo[13]=35'b10001_10001_11001_10101_10011_10001_10001 ; // N
tablo[14]=35'b01110_10001_10001_10001_10001_10001_01110 ; // O
tablo[15]=35'b11110_10001_10001_11110_10000_10000_10000 ; // P
tablo[16]=35'b01110_10001_10001_10001_10101_10011_01111 ; // Q
tablo[17]=35'b11110_10001_10001_11110_10100_10010_10001 ; // R
tablo[18]=35'b01111_10000_10000_11110_00001_00001_11110 ; // S
tablo[19]=35'b11111_00100_00100_00100_00100_00100_00100 ; // T
tablo[20]=35'b10001_10001_10001_10001_10001_10001_01110 ; // U
tablo[21]=35'b10001_10001_10001_10001_10001_01010_00100 ; // V
tablo[22]=35'b10001_10001_10001_10101_10101_11011_10001 ; // W
tablo[23]=35'b10001_10001_01010_00100_01010_10001_10001 ; // X
tablo[24]=35'b10001_10001_01010_00100_00100_00100_00100 ; // Y
tablo[25]=35'b11111_00001_00010_00100_01000_10000_11111 ; // Z
tablo[26]=35'b00000_00000_00000_00100_00000_00100_00000 ; // :
tablo[27]=35'b10000_10000_10000_10000_10000_00000_10000 ; // !
end
assign line=tablo[in];
// inner positions
assign xcell = (x>posx && x<(posx+26) && y>posy && y<(posy+36))?(x-posx):0; //
25x35
assign ycell = (x>posx && x<(posx+26) && y>posy && y<(posy+36))?(y-posy):0;
// rows
assign r1 = (ycell>0 && ycell<6)?1:0; // 5x5 squares
assign r2 = (ycell>5 && ycell<11)?1:0;
assign r3 = (ycell>10 && ycell<16)?1:0;
assign r4 = (ycell>15 && ycell<21)?1:0;
assign r5 = (ycell>20 && ycell<26)?1:0;
assign r6 = (ycell>25 && ycell<31)?1:0;
assign r7 = (ycell>30 && ycell<36)?1:0;
//columns
assign c1 = (xcell>0 && xcell<6)?1:0;
assign c2 = (xcell>5 && xcell<11)?1:0;
assign c3 = (xcell>10 && xcell<16)?1:0;
assign c4 = (xcell>15 && xcell<21)?1:0;
assign c5 = (xcell>20 && xcell<26)?1:0;
// Note that the first operation is bitwise and. For Example
// the semi column above (26. entry). It lights only when
// the monitoring is at c3&r4 or c3&r6
assign out = (((line[1:5] & {c1,c2,c3,c4,c5}) != 5'd0) && r1)
|| (((line[6:10] & {c1,c2,c3,c4,c5}) != 5'd0) && r2)
|| (((line[11:15] & {c1,c2,c3,c4,c5}) != 5'd0) && r3)
|| (((line[16:20] & {c1,c2,c3,c4,c5}) != 5'd0) && r4)
|| (((line[21:25] & {c1,c2,c3,c4,c5}) != 5'd0) && r5)
|| (((line[26:30] & {c1,c2,c3,c4,c5}) != 5'd0) && r6)
|| (((line[31:35] & {c1,c2,c3,c4,c5}) != 5'd0) && r7);
      endmodule

module letters (in, x, y, posx, posy, out);
input[4:0] in;
input[31:0] x, y, posx, posy;
output out;
wire[31:0] xcell, ycell;
wire[1:35] line;
```

```
reg[1:35] tablo[0:27];
wire r1,r2,r3,r4,r5,r6,r7,c1,c2,c3,c4,c5;
initial begin
tablo[0]=35'b01110_10001_10001_11111_10001_10001_10001 ; // A
tablo[1]=35'b11110_10001_10001_11110_10001_10001_11110 ; // B
tablo[2]=35'b01111_10000_10000_10000_10000_10000_01111 ; // C
tablo[3]=35'b11110_10001_10001_10001_10001_10001_11110 ; // D
tablo[4]=35'b11111_10000_10000_11110_10000_10000_11111 ; // E
tablo[5]=35'b11111_10000_10000_11110_10000_10000_10000 ; // F
tablo[6]=35'b01111_10000_10000_10111_10001_10001_01110 ; // G
tablo[7]=35'b10001_10001_10001_11111_10001_10001_10001 ; // H
tablo[8]=35'b11111_00100_00100_00100_00100_00100_11111 ; // I
tablo[9]=35'b00010_00010_00010_00010_10010_10010_01100 ; // J
tablo[10]=35'b10001_10010_10100_11000_10100_10010_10001 ; // K
tablo[11]=35'b10000_10000_10000_10000_10000_10000_11111 ; // L
tablo[12]=35'b10001_11011_10101_10101_10001_10001_10001 ; // M
tablo[13]=35'b10001_10001_11001_10101_10011_10001_10001 ; // N
tablo[14]=35'b01110_10001_10001_10001_10001_10001_01110 ; // O
tablo[15]=35'b11110_10001_10001_11110_10000_10000_10000 ; // P
tablo[16]=35'b01110_10001_10001_10001_10101_10011_01111 ; // Q
tablo[17]=35'b11110_10001_10001_11110_10100_10010_10001 ; // R
tablo[18]=35'b01111_10000_10000_11110_00001_00001_11110 ; // S
tablo[19]=35'b11111_00100_00100_00100_00100_00100_00100 ; // T
tablo[20]=35'b10001_10001_10001_10001_10001_10001_01110 ; // U
tablo[21]=35'b10001_10001_10001_10001_10001_01010_00100 ; // V
tablo[22]=35'b10001_10001_10001_10101_10101_11011_10001 ; // W
tablo[23]=35'b10001_10001_01010_00100_01010_10001_10001 ; // X
tablo[24]=35'b10001_10001_01010_00100_00100_00100_00100 ; // Y
tablo[25]=35'b11111_00001_00010_00100_01000_10000_11111 ; // Z
tablo[26]=35'b00000_00000_00000_00100_00000_00100_00000 ; // :
tablo[27]=35'b10000_10000_10000_10000_10000_00000_10000 ; // !
end
assign line=tablo[in];
assign xcell = (x>posx && x<(posx+16) && y>posy && y<(posy+22))?(x-posx):0;
//15x21
assign ycell = (x>posx && x<(posx+16) && y>posy && y<(posy+22))?(y-posy):0;
assign r1 = (ycell>0 && ycell<4)?1:0; // the squares are 3x3
assign r2 = (ycell>3 && ycell<7)?1:0;
assign r3 = (ycell>6 && ycell<10)?1:0;
assign r4 = (ycell>9 && ycell<13)?1:0;
assign r5 = (ycell>12 && ycell<16)?1:0;
assign r6 = (ycell>15 && ycell<19)?1:0;
assign r7 = (ycell>18 && ycell<22)?1:0;
assign c1 = (xcell>0 && xcell<4)?1:0;
assign c2 = (xcell>3 && xcell<7)?1:0;
assign c3 = (xcell>6 && xcell<10)?1:0;
assign c4 = (xcell>9 && xcell<13)?1:0;
assign c5 = (xcell>12 && xcell<16)?1:0;
assign out = (((line[1:5] & {c1,c2,c3,c4,c5}) != 5'd0) && r1)
|| (((line[6:10] & {c1,c2,c3,c4,c5}) != 5'd0) && r2)
|| (((line[11:15] & {c1,c2,c3,c4,c5}) != 5'd0) && r3)
|| (((line[16:20] & {c1,c2,c3,c4,c5}) != 5'd0) && r4)
|| (((line[21:25] & {c1,c2,c3,c4,c5}) != 5'd0) && r5)
|| (((line[26:30] & {c1,c2,c3,c4,c5}) != 5'd0) && r6)
|| (((line[31:35] & {c1,c2,c3,c4,c5}) != 5'd0) && r7);
      endmodule
```

```
module random (trigger, change1, change2, out);
input trigger, change1, change2;
output[6:0] out;
reg[6:0] seed=7'd1;
integer holder=0;
reg[2:0] count=2'd0;
wire change1, change2;
assign out=seed;
always @ (posedge trigger)
begin
if (change1) seed=1010101; // If 3 successive blocks at left, turn to right
if (change2) seed=0101010; // If 3 successive blocks at right, turn to left
if (!change1 && !change2)
begin
holder = 3215478521*seed+count; // X(n+1)=a*X(n)+b
seed = holder[6:0]; // modulus operation
if(count==7) count = 0;
else count = count+1;
end
end
      endmodule



module rotatedot (frame, ccwbutton, cwbutton, ccw, cw);
input frame, ccwbutton, cwbutton;
output ccw, cw;
reg ccwold=0, cwold=0, ccw=0, cw=0;
always @ (posedge frame)
begin
ccwold<=ccwbutton; // The previous values are held
cwold<=cwbutton;
if (ccwbutton && cwbutton) begin ccw<=0; cw<=0; end
if (!ccwbutton && cwbutton) begin ccw<=1; cw<=0; end
if (ccwbutton && !cwbutton) begin ccw<=0; cw<=1; end
if (!ccwbutton && !cwbutton) // If two of them is pushed together
begin
if (ccwold && cwold) begin ccw<=0; cw<=0; end
if (!ccwold && cwold) begin ccw<=0; cw<=1; end // If CW is unpressed
previously, it is
pressed lastly
if (ccwold && !cwold) begin ccw<=1; cw<=0; end
end
end
      endmodule



module statedecider (clk_frame, startbuttondb , pausebuttondb , collision,
state);
input clk_frame, startbuttondb , pausebuttondb , collision;
output[1:0] state;
reg[1:0] state=2'b00;
```

```
// state 00 starting screen
// state 01 game
// state 10 pause
// state 11 collision, end game

always @ (posedge clk_frame)
begin // The implementation of the flowchart
if (startbuttondb && state==2'b00) state=2'b01;
if (pausebuttondb && state==2'b01) state=2'b10;
if (collision && state==2'b01) state=2'b11;
if (pausebuttondb && state==2'b10) state=2'b01;
if (startbuttondb && state==2'b11) state=2'b00;
end
endmodule



module timecounter (frame, state, min1, min0, sec1, sec0, difficulty);
input frame;
input[1:0] state;
output[3:0] min1, min0, sec1, sec0, difficulty;
reg[3:0] min1=4'b0000, min0=4'b0000, sec1=4'b0000, sec0=4'b0000,
reg[3:0] difficulty=4'b0001;
reg[31:0] framecount=0;
wire frameout, sec0out, sec1out, min0out, min1out;
assign frameout = framecount==59;
assign sec0out = sec0==9;
assign sec1out = sec1==5;
assign min0out = min0==9;
assign min1out = min1==5;

// Time min1 min0 : sec1 sec0 (BCD Representation)
always @ (posedge frame)
begin
if (state==2'b01)
begin
      if (!frameout)
            framecount=framecount+1;
      else begin // If frame count is 59, frame count turns to 0
            framecount=0; // And sec0 increments ( if it's not 9)
      if(!sec0out) begin
            sec0=sec0+1;
      end
      else begin // If sec0 is 9
            sec0=0;
            if(!sec1out) begin
                  sec1=sec1+1;
            end
            else begin
                  sec1=0;
            if (!min0out) begin
                  min0=min0+1;
            end
            else begin
                  min0=0;
            if(!min1out) begin
                  min1=min1+1;
```

```
            end
            else begin
            min1=0;
            end
end
end
end
end
//Level Generation
if ((sec1==4'd3 || sec1==4'd0) && sec0==4'd0 && framecount==0 &&
difficulty!=4'd5)
difficulty=difficulty+4'd1;
end
if(state==2'b00) // If game turns to start screen, module restarts
begin
framecount=4'b0000;
min1=4'b0000;
min0=4'b0000;
sec1=4'b0000;
sec0=4'b0000;
difficulty=4'd1;
end
end
endmodule


module blockmaster (trigger, state,
newrequest1, newrequest2, newrequest3, newrequest4,
yend1, yend2, yend3, yend4,
enable1, enable2, enable3, enable4);
input trigger, newrequest1, newrequest2, newrequest3, newrequest4,
yend1, yend2, yend3, yend4;
input[1:0] state;
output enable1, enable2, enable3, enable4;
reg enable1=0, enable2=0, enable3=0, enable4=0, starting=1;
reg newrequest1local =0, newrequest2local =0, newrequest3local =0,
newrequest4local =0;
always @ (posedge trigger) begin
if(state==2'b01) // It is not needed but just in case
begin //(in pause or endgame there is no yend or newrequestlocal)
if (starting)
begin enable1<=1; starting<=0; end // only in the begining, it is needed to
start
if (newrequest1local && !enable2) // If there is need for new block and if
there is
begin enable2<=1; newrequest1local <=0; end // available new block, a new
block is
if (newrequest2local && !enable3) // initiated
begin enable3<=1; newrequest2local <=0; end
if (newrequest3local && !enable4)
begin enable4<=1; newrequest3local <=0; end
if (newrequest4local && !enable1)
begin enable1<=1; newrequest4local <=0; end
if (yend1 && enable1) enable1<=0;// if a block comes to the end
if (yend2 && enable2) enable2<=0;// it is removed
if (yend3 && enable3) enable3<=0;
if (yend4 && enable4) enable4<=0;
```

```
if (!newrequest1local && newrequest1 && !enable2) newrequest1local <=1; //
Purpose is to
if (!newrequest2local && newrequest2 && !enable3) newrequest2local <=1; //
initiate only
if (!newrequest3local && newrequest3 && !enable4) newrequest3local <=1; // one
block because
if (!newrequest4local && newrequest4 && !enable1) newrequest4local <=1; //
newrequest's are
// continuously high for a time
end
if(state==2'b00) // When a new game starts, removes all the blocks and
begin // leaves a trigger for the first block to start the flow again
enable1<=0; enable2<=0; enable3<=0; enable4<=0; starting<=1;
end
end
      endmodule



module behavioralblock (x, y, clk, enable, state, instructionin , difficulty,
posx, newrequest, yend, score, fade,out);
input[31:0] x, y;
input clk, enable;
input[1:0] state;
input[3:0] difficulty;
input[6:0] instructionin ;
output newrequest, yend, score, fade, out;
output[31:0] posx;
wire out;
integer posx=0, posy=0, mode10posx;
reg yend=0, newrequest=0, score=0, mode10init=1, mode10dir=0, mode01init=1,
fade=0;
reg[6:0] instruction=0;
reg[8:0] angle=0;
integer sinusoidal[0:45];
integer tangential[0:45];
wire[31:0] toplimity, downlimity, toplimitx, downlimitx, sinblock1, cosblock1,
wire[31:0] tanblock1a, tanblock1b, factor1, factor2, factor3, factor4,
wire[31:0] factor5, factor6, factor7, factor8, sizex, sizey;
wire[6:0] definition;
reg[6:0] definitionused =0;
wire[3:0] speed;
// Representative Variable Generating Block
assign sinblock1 = (angle<46)?sinusoidal[45-angle]:(angle>45 &&
angle<91)?sinusoidal[angle-45]
:(angle>90 && angle<136)?(2048-sinusoidal[135-angle])
:(angle>135 && angle<180)?(2048-sinusoidal[angle-135]):31'bx;
assign cosblock1 = (angle<46)?sinusoidal[angle]:(angle>45 && angle<91)?(2048-
sinusoidal[90-angle])
:(angle>90 && angle<136)?(2048-sinusoidal[angle-90])
:(angle>135 && angle<180)?sinusoidal[180-angle]:31'bx;
assign tanblock1a = (angle<46)?tangential[angle]:(angle>45 && angle<91)?(-
1*tangential[90-angle])
:(angle>90 && angle<136)?(tangential[angle-90])
:(angle>135 && angle<180)?(-1*tangential[180-angle]):31'bx;
assign tanblock1b = (angle<46)?(-1*tangential[45-angle])
:(angle>45 && angle<91)?(tangential[angle-45])
```

```
:(angle>90 && angle<136)?(-1*tangential[135-angle])
:(angle>135 && angle<180)?(tangential[angle-135]):31'bx;
assign factor1 = sizey*sinblock1;
assign factor2 = sizey*cosblock1;
assign factor3 = sizex*sinblock1;
assign factor4 = sizex*cosblock1;
assign factor5 = (((tanblock1a[31]==1)?(-1):1)*((tanblock1a[31]==1)?(-
tanblock1a):tanblock1a)
*(-x + posx -((factor1[31] ==1)?(-((-factor1) >> 10)):(factor1 >> 10)) +
sizey));
assign factor6 = ((tanblock1a[31]==1)?(-1):1)*((tanblock1a[31]==1)?(-
tanblock1a):tanblock1a)
*(-x + posx + ((factor1[31] ==1)?(-((-factor1) >> 10)):(factor1 >> 10)) -
sizey);
assign factor7 = ((tanblock1b[31]==1)?(-1):1)*((tanblock1b[31]==1)?(-
tanblock1b):tanblock1b)
*(-x + posx + ((factor4[31] ==1)?(-((-factor4) >> 10)):(factor4 >> 10)) -
sizex);
assign factor8 = ((tanblock1b[31]==1)?(-1):1)*((tanblock1b[31]==1)?(-
tanblock1b):tanblock1b)
*(-x + posx - ((factor4[31] ==1)?(-((-factor4) >> 10)):(factor4 >> 10)) +
sizex);
assign toplimity = ((factor5[31] == 1)?(-((-factor5) >> 10)):(factor5 >> 10))
+ posy
- ((factor2[31] ==1)?(-((-factor2) >> 10)):(factor2 >> 10))
+sizey;
assign downlimity = ((factor6[31] == 1)?(-((-factor6) >> 10)):(factor6 >> 10))
+ posy
+ ((factor2[31] ==1)?(-((-factor2) >> 10)):(factor2 >> 10))
- sizey;
assign toplimitx = ((factor7[31] == 1)?(-((-factor7) >> 10)):(factor7 >> 10))
+ posy
- ((factor3[31] ==1)?(-((-factor3) >> 10)):(factor3 >> 10))
+ sizex;
assign downlimitx = ((factor8[31] == 1)?(-((-factor8) >> 10)):(factor8 >> 10))
+ posy
+ ((factor3[31] ==1)?(-((-factor3) >> 10)):(factor3 >> 10))
- sizex;
assign out = enable && (angle!=0 && angle!=45 && angle!=90 && angle!=135) //
at these angles on
of the tangents
?(y>((toplimity[31]==0)?toplimity:0) ^ y>((downlimity[31]==0)?downlimity:0)
)// is infinite.
&& (y>((toplimitx[31]==0)?toplimitx:0) ^ y>((downlimitx[31]==0)?downlimitx:0)
)
: (angle==0 || angle==90) ?
( x<posx+sizex && x>posx-sizex && (y<(posy+sizey) && ((posy+sizey)>>31)==0)
&& (y>(posy-sizey) || ((posy-sizey)>>31)==1))
: (angle==45 || angle==135) ?
( x<posx+sizey && x>posx-sizey && (y<(posy+sizex) && ((posy+sizex)>>31)==0)
&& (y>(posy-sizex)|| ((posy-sizex)>>31)==1))
:0;
// End of Representative Variable Generating Block
// Random Utilization for only the Behavior of the block
assign definition[6:5]=(difficulty<2)?2'b00
:(difficulty<3 && instruction[6:5]==2'b01)?2'b01
:(difficulty<3)?2'b00
```

```
:(difficulty<4 && instruction[6:5]==2'b11)?2'b00
:instruction[6:5];
// Size Deciding
assign sizex= (definitionused [4]==1'b0)?30:70;
assign sizey= (definitionused [4:3]==2'b00)?30:15;
// Speed adjusting according to level
assign speed= (difficulty<3)?4'd2:(difficulty<5)?4'd3:4'd4;
initial begin
sinusoidal[0]=2048;
sinusoidal[1]=2047;
sinusoidal[2]=2046;
sinusoidal[3]=2042;
sinusoidal[4]=2038;
sinusoidal[5]=2032;
sinusoidal[6]=2026;
sinusoidal[7]=2018;
sinusoidal[8]=2008;
sinusoidal[9]=1998;
sinusoidal[10]=1986;
sinusoidal[11]=1973;
sinusoidal[12]=1959;
sinusoidal[13]=1944;
sinusoidal[14]=1928;
sinusoidal[15]=1911;
sinusoidal[16]=1892;
sinusoidal[17]=1873;
sinusoidal[18]=1852;
sinusoidal[19]=1831;
sinusoidal[20]=1808;
sinusoidal[21]=1785;
sinusoidal[22]=1761;
sinusoidal[23]=1735;
sinusoidal[24]=1709;
sinusoidal[25]=1682;
sinusoidal[26]=1654;
sinusoidal[27]=1626;
sinusoidal[28]=1597;
sinusoidal[29]=1567;
sinusoidal[30]=1536;
sinusoidal[31]=1505;
sinusoidal[32]=1473;
sinusoidal[33]=1440;
sinusoidal[34]=1408;
sinusoidal[35]=1374;
sinusoidal[36]=1340;
sinusoidal[37]=1306;
sinusoidal[38]=1272;
sinusoidal[39]=1237;
sinusoidal[40]=1202;
sinusoidal[41]=1167;
sinusoidal[42]=1131;
sinusoidal[43]=1095;
sinusoidal[44]=1060;
sinusoidal[45]=1024;
tangential[0]=0;
tangential[1]=36;
tangential[2]=72;
```

```
tangential[3]=108;
tangential[4]=144;
tangential[5]=181;
tangential[6]=218;
tangential[7]=255;
tangential[8]=294;
tangential[9]=333;
tangential[10]=373;
tangential[11]=414;
tangential[12]=456;
tangential[13]=499;
tangential[14]=544;
tangential[15]=591;
tangential[16]=640;
tangential[17]=691;
tangential[18]=744;
tangential[19]=800;
tangential[20]=859;
tangential[21]=922;
tangential[22]=989;
tangential[23]=1060;
tangential[24]=1137;
tangential[25]=1220;
tangential[26]=1311;
tangential[27]=1409;
tangential[28]=1518;
tangential[29]=1639;
tangential[30]=1774;
tangential[31]=1926;
tangential[32]=2100;
tangential[33]=2300;
tangential[34]=2534;
tangential[35]=2813;
tangential[36]=3152;
tangential[37]=3571;
tangential[38]=4107;
tangential[39]=4818;
tangential[40]=5807;
tangential[41]=7286;
tangential[42]=9743;
tangential[43]=14644;
tangential[44]=29324;
tangential[45]=32'dx;
end
always @ (posedge clk)
begin
// Random Utilization For Size and Horizontal Position
definitionused [4:0]=(((definitionused [6:5]==2'b00 || definitionused
[6:5]==2'b11)
&& instruction[4]==1'b1) || definitionused [6:5]==2'b10)
?({instruction[4:3],2'b10,instruction[0]})
:((definitionused [6:5]==2'b00 || definitionused [6:5]==2'b11) &&
instruction[4]==1'b0)
?instruction[4:0]
:5'b10000;
if (enable && !yend)
begin
```

```
// This block increments or decrements the parameters of the block.
if(state==2'b01) // These operations only done in gameplay state. If State is
pause
begin // it neither does these operations nor resets the parameters.
if(definitionused [6:5]!=2'b10) // The horizontal allignment for all except
begin // the horizontally moving type
if(definitionused [2:1]==2'b00) posx<=320;
else if(definitionused [2:1]==2'b01 || definitionused [2:0]==3'b100)
posx<=265;
else posx<=375;
end
else // Horizontally moving block
begin
if (mode10init)
begin
if(definitionused [0]) begin mode10posx<=241; mode10dir<=1; mode10init<=0; end
else begin mode10posx<=399; mode10dir<=0; mode10init<=0; end
end
if (!mode10init && mode10dir)
begin
if (mode10posx>(409-speed)) mode10dir<=0;
mode10posx<=mode10posx+speed;
end
if (!mode10init && !mode10dir)
begin
if (mode10posx<(231+speed)) mode10dir<=1;
mode10posx<=mode10posx-speed;
end
if (mode10posx>375) posx<=375;
else if (mode10posx<265) posx<=265;
else posx<=mode10posx;
end
if(definitionused [6:5]==2'b01) // Rotating mode
begin
if(mode01init) begin angle<=156; mode01init=0; end
else begin
if(angle>(179-speed)) angle <= (angle+speed-180);
else angle<=angle+speed;
end
end
else angle<=0;
posy<=posy+speed; // Vertical movement
if (posy>180) newrequest<=1; // Interactions with outside
if (posy>250 && definitionused [6:5]==2'b11) fade<=1;
if (posy>470) score<=1;
if (posy>520) yend<=1;
end
end
else begin
posx<=0; posy<=0;
end
if (!enable) // enable variable controlled by 'Blockmaster' module.
begin // When it is zero, everything resets.
instruction<=instructionin ;
definitionused [6:5]<=definition[6:5];
yend<=0;
newrequest<=0;
```

```
fade<=0;
score<=0;
posx<=0;
posy<=0;
mode10init<=1;
mode01init<=1;
angle<=0;
end
end
      endmodule



module scorecounter (frame, state, score1, score2, score3, score4, s3, s2, s1,
s0);
input frame, score1, score2, score3, score4;
input[1:0] state;
output[3:0] s3, s2, s1, s0;
reg[3:0] s3=0, s2=0, s1=0, s0=0;
reg score1done=0, score2done=0, score3done=0, score4done=0;
wire s0out, s1out, s2out, s3out;
assign s0out = s0==9;
assign s1out = s1==9;
assign s2out = s2==9;
assign s3out = s3==9;
// Score; s3 s2 s1 s0 (BCD representation)
always @ (posedge frame)
begin
if (state==2'b01 && ((score1 && !score1done) || (score2 && !score2done)
|| (score3 && !score3done) || (score4 && !score4done)))
begin
if(~s0out) s0<=s0+1; // If s0 is 9 it turns to 0 and increments s1
else begin
s0=0;
if(~s1out) s1<=s1+1;
else begin
s1=0;
if (~s2out) s2<=s2+1;
else begin
s2=0;
if(~s3out) s3<=s3+1;
else begin
s3<=0;
end
end
end
end
if (score1 && !score1done) score1done<=1; // scorexdone aims not to
if (score2 && !score2done) score2done<=1; // increment continuously because
scorex
if (score3 && !score3done) score3done<=1; // stays high for a period of time
if (score4 && !score4done) score4done<=1; // It should increment a single time
end
if(!score1) score1done<=0;
if(!score2) score2done<=0;
if(!score3) score3done<=0;
if(!score4) score4done<=0;
```

```
if (state==2'b00) // Resets while turning to Start screen
begin
s3<=0;
s2<=0;
s1<=0;
s0<=0;
score1done<=0;
score2done<=0;
score3done<=0;
score4done<=0;
end
end
      endmodule



module highscore (clk_frame,s3,s2,s1,s0,shigh3,shigh2,shigh1,shigh0,
highhappened );
input clk_frame;
input[3:0] s3,s2,s1,s0;
output[3:0] shigh3,shigh2,shigh1,shigh0;
output highhappened ;
reg[3:0] shigh3=4'd0,shigh2=4'd0,shigh1=4'd0,shigh0=4'd0;
wire act;
reg highhappened =0;
assign act=({s3,s2,s1,s0}>{shigh3,shigh2,shigh1,shigh0});// Whether the
current score
// is greater or not
always @ (posedge clk_frame)
begin
if(act) // if high score
begin
highhappened =1'b1; // according to this, there will be "High Score!" text or
not
shigh3=s3; // The high score held
shigh2=s2;
shigh1=s1;
shigh0=s0;
end
if (s3==0 && s2==0 && s1==0 && s0==0) highhappened =1'b0; // restart for the
new game
// the condition satisfies only
// the state turns to start screen
end
      endmodule
```